IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 11 September 2026, accepted 28 September 2026, date of publication 2 October 2026, date of current version 8 October 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3739345

RESEARCH ARTICLE

Reducing Hallucinated False Positives in LLM-Based Vulnerability Detection via Multi-Signal Focused Verification

CASPER CHEN
Cevanex, New Taipei City 235, Taiwan
e-mail: casperchen@cappuccinoo.com

ABSTRACT Large language models used for vulnerability detection hallucinate, producing false positives that bury genuine vulnerabilities. We present a multi-signal focused verification stage that combines four signals—identity, source code, concern, and proof-of-concept (PoC) verdict—and admits the PoC verdict in three states, so that a PoC that fails to compile is withheld rather than counted as counterevidence. The signals are non-redundant, with pairwise agreement bounded away from both chance and interchangeability (κ between -0.05 and 0.44), so one can correct another. Across four benchmarks in three languages, over five independent full-pipeline runs, the stage removes 33.7% to 75.8% of shipped findings at the precision-first operating point, and 54.6% to 64.6% in the main setting (qwen3-coder detection). Confirmation of true positives rests on source code and concern, divided by language—concern dominates on Solidity, source code on Python and JavaScript—while the PoC verdict suppresses shipped findings at a recall cost. The stage reduces shipped findings in all 36 detector–verifier–benchmark configurations, so the architecture transfers; the magnitude varies with the breadth of detection, the availability of the PoC verdict, and the verifier. Freezing three candidate pools and varying only the verifier moves the benchmark-averaged reduction by 7 to 27 percentage points; the strongest verifier removes the most volume but retains the fewest detector-surfaced vulnerabilities (52.6% against 74.4% and 72.3%), so the choice of verifier is a precision–recall trade-off.

INDEX TERMS False positive reduction, large language models, software security, vulnerability detection.

I. INTRODUCTION

As large language models (LLMs) continue to advance, engineers have increasingly adopted them to detect vulnerabilities in their workflows. This use, however, faces a critical problem: LLMs tend to produce a high false positive rate, far exceeding the rate of true positives [1]. Independent evaluations report the same pattern: in a recent benchmark, off-the-shelf LLM detectors flagged seven and ten of the ten safe contracts [2], and unvalidated prompting has been characterised as yielding "unverified vulnerability speculations with high false-positive rates" [3]. Genuine vulnerabilities are buried among false alarms, forcing engineers to spend considerable time sifting through findings.

The associate editor coordinating the review of this manuscript and approving it for publication was Alba Amato.

Static analysis tools are useful but share the same problem: they produce many false positives [4], require extensive filtering rules to be usable, and have no strategy for finding new types of vulnerabilities. LLM-based detection is itself prone to hallucinated false positives; among existing work, some perform only classification over a narrow class of vulnerabilities [5], while others verify only through static reasoning within a single domain [6]. A separate line of work uses an LLM to filter the warnings of static analysers [7], [8], [9], [10], [11], but it targets the analyser's over-approximation rather than the model's own hallucination, and none of these filters, as evaluated, uses execution evidence. The methods we currently have are therefore not enough to solve this problem.

To address this, we present a multi-signal focused verification stage that combines four signals: identity, PoC verdict,

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

concern, and source code. The idea behind using multiple signals is that they cover one another's blind spots. The signals are non-redundant: each carries evidence the others do not, so one can correct another rather than reinforce a shared mistake. Since hallucination is largely caused by the distraction of a large context, we give the LLM the source code and concern produced by the detection stage, together with the outcome of PoC execution, so as to keep the model focused and provide concrete evidence—thereby reducing its hallucination rate. Prior LLM-based auditors and warning inspectors report the same sensitivity to context size: part of their false positives trace to inputs beyond roughly 200 lines [12], to long functions with cascaded constraints [9], and to routines several hundred lines long [7].

As described above, this work makes five contributions. First, we propose a multi-signal focused verification stage whose four signals are non-redundant, a property we measure directly (κ between -0.05 and 0.44). Second, we use PoC-based execution as the source of the execution signal, and admit its verdict in three states: a PoC that triggers the reported behaviour confirms the finding, one that compiles but does not trigger is admitted as counterevidence, and one that fails to compile carries no information and is withheld. Withholding the third state is what prevents an absent signal from being misread as evidence of absence. Third, we evaluate our approach across four benchmarks with documented vulnerabilities and five independent runs of the full pipeline per configuration, removing 33.7% to 75.8% of shipped findings at the precision-first operating point. Fourth, we ablate the complete power set of the four signals and quantify each signal's average marginal contribution. Fifth, the architecture transfers across model choices: all 36 configurations we evaluate—three detection models, three verification models, and four benchmarks—yield a positive reduction, so the effect is not confined to one model pairing. Isolating the verification stage by freezing three independently produced candidate pools and varying only the verifier moves the benchmark-averaged alert-count reduction by 7 to 27 percentage points; the strongest verifier removes the most volume but retains the fewest detector-surfaced vulnerabilities, so the choice of verifier is a precision–recall trade-off.

II. RELATED WORK

A. STATIC ANALYSIS FOR VULNERABILITY DETECTION

Static analysis tools such as Slither [13] and Semgrep [14] detect vulnerabilities through pattern matching and dataflow analysis. While effective at scale, they are well known to suffer from high false positive rates [15], as their predefined rules cannot fully capture contextual semantics. Out of the box they also miss most known vulnerabilities: four popular SAST tools detected 11.2–26.5% of 170 real Java vulnerabilities, and Semgrep reached 44.7% only after its rule set was extended [16]; in a Solidity benchmark, Slither and Mythril raised no false alarms on the ten safe contracts but recovered only 30 and 54 of the 100 labelled vulnerabilities [2].

B. LLM-BASED VULNERABILITY DETECTION AND EXPLOIT GENERATION

Recent work applies LLMs to vulnerability detection and automated exploit generation [17]. A systematic review that re-ran five LLM-based smart-contract detectors against Slither and Mythril concluded that they are "not ready to replace more traditional tools" [18]. Multi-agent and plan–execute auditors [19], [20], [21], rule-by-rule ERC-compliance checking [12], and prover-backed property or invariant generation [22], [23] all add a check on the model's output, but the check is static; A1 [3] validates LLM-generated exploits by executing them on forked chain state, yet its single verdict is a binary profitability oracle rather than an adjudication of individual findings. Among these, VulnSage [24] similarly employs a multi-agent LLM design, but its detection relies primarily on static program analysis; the LLM is used mainly in a later stage to generate and iteratively refine exploits (AEG), using execution traces and runtime errors in an exploit-centered reflection loop that both confirms findings and reasons about whether an alert is a false positive, thereby reducing the false positives inherent to static analysis. Our approach differs in two ways. First, while VulnSage uses the LLM mainly to generate exploits after static detection, our pipeline combines two complementary inputs to the LLM: a static scanner (Semgrep) supplies candidate findings, and a bidirectional call graph supplies the surrounding cross-function context. The LLM then reasons over this combined input to assess each candidate, rather than serving only as a back-end exploit generator. Second, because an LLM operating over broad context is itself prone to hallucinated false positives—a different source of error from static-analysis false alarms—we introduce a verification layer (Layer 5.5) that adjudicates each candidate by combining multiple signals (identity, concern, PoC verdict, and source code), rather than relying on the rigid pattern matching of traditional tools.

C. CONTEXT-AWARE AND VERIFICATION APPROACHES

Another line of work narrows the code given to the LLM through static slicing. LLMxCPG [5] fine-tunes one model to generate code-property-graph queries that extract vulnerability-relevant slices, and a second to classify each slice as vulnerable or safe; the slicing serves primarily to fit the code within the detection model's context window. PriceSleuth [6] couples Slither-based dependency and taint slicing with LLMs in a detect-then-verify pipeline for price-manipulation vulnerabilities, where its verification stage statically checks whether a manipulated price is actually exploited. Our approach differs in three ways. First, rather than statically slicing the code in advance, we use a call graph to assemble cross-function context around each candidate, preserving the surrounding caller/callee structure rather than

reducing the code to an isolated slice. Second, both methods are narrow in scope—LLMxCPG targets memory-safety CWEs and PriceSleuth targets Web3 price manipulation—whereas our analysis is not confined to a single vulnerability domain. Third, and most importantly, neither verifies findings against the kind of error we target: LLMxCPG performs no verification beyond classification, and PriceSleuth's verification reasons statically about exploitability. In contrast, our verification layer combines multiple signals—including actual PoC execution—specifically to reduce the hallucinated false positives introduced when an LLM detects over broad context.

III. METHOD

A. OVERVIEW

We present a multi-stage pipeline for vulnerability detection that combines LLM reasoning with execution-based verification. The central problem we address is that LLMs, when reasoning over large or noisy code context, frequently report vulnerabilities that do not exist—a failure mode we refer to as *hallucinated false positives*. Our approach mitigates this through a multi-signal verification stage applied after an initial detection phase.

The pipeline proceeds as follows. First, a detection stage identifies candidate vulnerabilities across the target codebase. We construct a bidirectional call graph linking each function to both its callers and callees, and for each candidate finding we use it to assemble the relevant cross-function context—entry points, callees, and related state—into a single bundle passed to the LLM for analysis. While this cross-function context enables detection of vulnerabilities that span multiple functions, its breadth also makes the model prone to hallucinated false positives. Each candidate is then passed to a multi-signal verification stage (Section III-D), which re-examines the finding by combining multiple signals—such as the isolated source of the relevant function and the result of PoC-based execution—to decide whether the reported vulnerability is genuine.

Figure 1 illustrates this flow. The remainder of this section details each stage. Our primary contribution lies in the verification stage, which reduces hallucinated false positives without discarding genuine vulnerabilities.

B. SYSTEM ARCHITECTURE

Our system is a multi-stage pipeline organised into two phases: a detection phase that surfaces candidate vulnerabilities across the codebase, and a verification phase that adjudicates each candidate. Figure 1 gives an overview.

The detection phase combines static tooling with LLM reasoning. It first selects relevant source files and builds a bidirectional call graph of the codebase, linking every function to both its callers and callees. A static scanner provides an initial set of candidate findings, which an LLM-based filter triages to remove obvious non-issues. The surviving candidates then undergo LLM deep analysis, in which the model examines each finding against its surrounding call context and, where applicable, produces a proof-of-concept (PoC) exploit. A separate execution step compiles and runs each PoC to obtain an objective signal of whether the reported behaviour can actually be triggered. Finally, the LLM performs a set of proactive scans, each targeting a specific vulnerability category—such as reentrancy, access-control gaps, or flash-loan oracle exposure—and surfaces findings for that category by reasoning over the relevant functions and their call chains. This complements the per-finding deep analysis above by catching patterns that line-level signature scanners do not match.

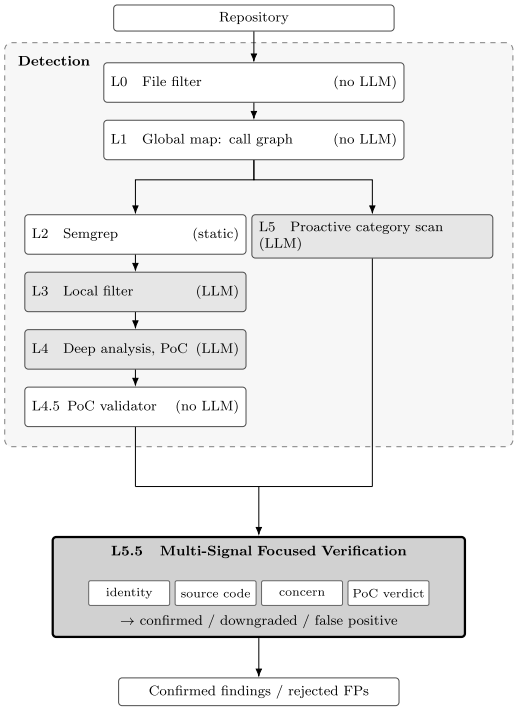


FIGURE 1. Overview of the pipeline. The detection phase (Layers 0–5) combines static analysis and LLM reasoning to surface candidate concerns; the multi-signal focused verification stage (Layer 5.5) adjudicates each concern into one of three verdicts—confirmed, downgraded, or false positive. False positives are clamped to informational severity and excluded from the shipped report. Shaded boxes denote LLM-driven stages.

The output of the detection phase is a set of candidate concerns, each accompanied by the relevant source and, where available, a PoC execution verdict. These candidates are passed to the verification phase—the focus of this paper—which combines multiple signals to decide whether each concern is genuine. We detail the detection stage in Section III-C and the verification stage in Section III-D.

C. CALL-GRAPH-GUIDED DETECTION

The detection stage is responsible for surfacing candidate vulnerabilities across the target codebase. Rather than analysing each function in isolation, we assemble cross-function context for each candidate finding using the bidirectional call graph constructed earlier. Starting from a flagged function,

the system performs a bidirectional breadth-first traversal: upward through the reverse call graph to recover the entry points that can reach the function, and downward through the call graph to expand the chain of callees it invokes. Because the call graph is keyed by function rather than by file, this traversal crosses file boundaries, pulling in the bodies of relevant functions wherever they are defined. The traversal also gathers the state variables the function depends on and the relevant access-control context. The resulting function bodies are assembled, within a fixed token budget, into a single context bundle that is passed to the LLM for one-shot deep analysis. This call-graph-guided assembly is well suited to vulnerabilities that span multiple functions or files, since the bundle can expose data and control flow across call boundaries that a single-function analysis would miss—for example, a privileged call made before a required state check, where the caller and the callee are defined in different contracts. However, the same breadth carries a cost. Because the bundle aggregates a large amount of surrounding context, the model's judgment can become detached from any single concrete piece of code, and it becomes prone to reporting vulnerabilities that are not borne out by the actual implementation—the hallucinated false positives that the verification stage is designed to reduce.

D. MULTI-SIGNAL FOCUSED VERIFICATION

The verification layer receives four signals: the identity, the PoC verdict, the concern raised, and the source code. The identity pins the concern to a specific file and function. The concern is the claim to be verified; the remaining two are the evidence provided to the LLM.

LLMs can hallucinate and produce false positives. Why, then, should this stage reduce the false positive rate? Because detection and verification fail for different reasons. During detection the model hallucinates because it has no concrete evidence to ground its search, and the codebase is too large for it to focus on the specific vulnerable function. Verification addresses both conditions, and the reason is the evidence. The concern explains why a vulnerability might exist. When it is informative, the PoC verdict shows how the code actually behaves at run time and whether that behaviour matches what the concern claims; because the verdict is an observation rather than an inference, the model is far less able to override it with plausible-sounding reasoning, and is therefore much less likely to hallucinate it away. The source code gives the model a complete view of the relevant function, and by confining the model's attention to that function it removes the broad context that induced hallucination during detection. No single signal is sufficient: the PoC verdict is decisive only when execution is conclusive, while the concern identifies what to check but cannot settle it.

Their combination covers the blind spots of each. This works because the signals are non-redundant. We measure pairwise agreement between the verdicts each signal would support in isolation, and Cohen's κ stays between -0.05 and 0.44, so one signal can correct another rather than reinforce a shared mistake. The two ends match the roles the signals play. Agreement is highest between the source code and the concern on all four benchmarks, which is expected, since both are static evidence read from the same artifact. It is lowest between the source code and the PoC verdict, falling to $\kappa = -0.046$ on PyGoat: static and execution evidence are nearly unrelated, which is exactly what makes them complementary. How the PoC verdict is admitted therefore matters as much as whether it is available: an uninformative verdict must not be allowed to act as counterevidence. We do not synthesise a PoC within the broad, mixed context of a cross-function candidate. There it would be constructed out of the very reasoning under scrutiny and would cease to be evidence external to that reasoning; such a PoC also tends to fail to compile or to reach the target path, and a failed verdict is easily misread as evidence that no vulnerability exists, discarding genuine findings (the same failure mode we describe for fuzzing in Section III-E). When a candidate instead references a function for which a cleanly constructed PoC already exists, we reuse that verdict, so that the execution evidence remains external to the reasoning it is used to check. The PoC verdict thus enters the verifier only when it carries informative content; otherwise it is omitted or explicitly flagged as inconclusive, with the verifier directed to rely on source-based judgment.

With these signals together, the model has a clear view of the candidate vulnerability and can determine whether the concern is a false positive, a downgraded finding, or a genuine vulnerability.

E. DESIGN RATIONALE: POC OVER FUZZING

The execution signal used during verification could, in principle, be produced by fuzzing. Our initial design did exactly this, generating fuzz harnesses and exercising them to confirm reported vulnerabilities. In practice, we found this approach unreliable for two reasons. First, the automatically generated harnesses frequently failed to compile or did not exercise the reported code path, so the test result reflected the quality of the harness rather than the presence of the vulnerability. Second, even when a harness was valid, random input search rarely reached vulnerabilities that require a specific state or call sequence to trigger; such cases passed silently and were misread as evidence that no vulnerability existed.

We attribute this to a mismatch between the tool and the task. Fuzzing is designed for *discovery*—searching a large input space for unknown faults—whereas verification asks a narrower question: does a *specific, already-localised* concern actually hold? This calls for a targeted trigger rather than a broad search. We therefore use PoC-based execution, in which a PoC is constructed to exercise the specific reported vulnerability directly. When the PoC compiles and triggers the reported behaviour, it provides evidence that the concern is genuine and the finding is confirmed.

IV. EVALUATION

A. EXPERIMENTAL SETUP

We use four benchmarks as our test projects, spanning three different languages. The first is Damn Vulnerable DeFi [25], a benchmark of intentionally vulnerable Solidity contracts. The second is NodeGoat [26], an OWASP benchmark of vulnerable Node.js (JavaScript) applications. The third is PyGoat [27], an OWASP benchmark of vulnerable Django (Python) applications. The fourth is SmartBugs [28], a benchmark of 143 vulnerable Solidity contracts. On SmartBugs the PoC signal is structurally unavailable: synthesis was attempted for 2994 candidates across all runs, but none compiled (Table 1), since the contracts use `^0.4.x` pragmas and ship without a buildable harness.

Our pipeline uses different models at different stages. In the main experiments, the detection stage uses qwen3-coder, while the verification stage uses NVIDIA-Nemotron-3-Super-120B-A12B; this division lets detection run cheaply at scale while verification applies a stronger model where judgment quality matters most. In RQ7a we additionally compare gemma-4-31B, qwen3-coder, and Nemotron as the detection model, holding verification fixed to Nemotron; RQ7b inverts this, freezing the candidate pool and varying the verification model instead. All stages use the inference settings in Table 2. We repeat every configuration because LLM outputs vary across runs even at low or zero temperature [29], [30], [31], [32], and we record every decoding parameter because incomplete reporting of such settings has been identified as a major obstacle to reproducing LLM-based software-engineering results [33].

Our ablation covers the complete power set of the four signals: all $2^4 - 1 = 15$ non-empty subsets, together with a *baseline* in which verification is disabled and detection findings are shipped directly (see Figure 1). The five subsets we refer to most often are *full*, with all four signals, and the four leave-one-out configurations—*no-identity*, *no-concern*, *no-source*, and *no-PoC*—each removing a single signal. On SmartBugs the power set reduces to $2^3 - 1 = 7$ subsets, since configurations containing the PoC signal are identical to those without it. Every configuration is run five times as an independent full pipeline, not as a repeated verification pass over fixed detection output, and is reported at two operating points: a precision-first threshold and a recall-first threshold (Section IV-B). For each benchmark, ground truth is taken from its documented or intentionally introduced vulnerabilities, and recall is measured in a mechanism-aware manner (Section IV-C).

B. FALSE POSITIVE CRITERION

True positive (TP) is defined by the standard vulnerability of the test project: a finding is counted as TP only when it fully matches the documented vulnerability, including its file and vulnerability type. False positive (FP) is defined as a finding whose reported vulnerability either cannot be exploited or does not exist. False positive of type (FP_t) refers to a finding that identifies a genuinely vulnerable location but misclassifies its vulnerability type; following [34], FP_t is counted as FP. Precision is computed as precision $=$ TP/(TP + FP), where FP includes FP_t.

The *baseline* against which we measure reduction is the set of candidate findings that reach the verification layer but are not adjudicated by it. It is not the raw detector output: the detector already rejects some of its own candidates, marking them `Confirmed: no`, and findings clamped to informational severity are dropped as well. Both filters are applied to the baseline, so the only difference between the baseline and a verified configuration is the verification verdict itself. This matters because the detector's self-rejection is substantial and varies widely across configurations, from 8.4% of candidates on PyGoat under qwen3-coder detection to 64.7% on SmartBugs under the same model (Table 3). Measuring against the post-self-rejection volume therefore attributes the reported reduction to the verification stage rather than to the detector.

A finding is *not* shipped when any of four conditions holds. First, the deep-analysis layer self-rejected it, marking its verdict `Confirmed: no`. Second, every verification verdict attached to it is `false_positive`. Third, at the medium reporting threshold only, every verdict is either `false_positive` or `downgraded` with no `confirmed` among them. Fourth, its severity is clamped to informational. The medium threshold corresponds to the precision-first operating point and the low threshold to the recall-first point; the two share the same verdicts and differ only in which of them are reported. Applying these four rules to the recorded findings reproduces the shipped count exactly in all 964 run-configurations we report, so every percentage in Section IV-E can be recomputed from the underlying counts.

The quantity we report as the main result is an *alert-count reduction*: the fraction of shipped findings that verification removes, counted over all findings regardless of label, so that it includes the genuine findings the stage withholds. It is neither a false positive rate nor a reduction in false positives alone, and we report those two quantities separately. The reduction in false positives proper—the share of the unlabelled findings in the verifier's pool that it withholds—is given for every detector and benchmark in Table 6, next to the share of labelled findings withheld. The conventional rate is computed over the pool of candidates that reach the verifier, that is, the baseline of the previous paragraph, by treating the verifier's ship or no-ship decision as the prediction and whether the candidate matches a documented vulnerability as the reference. Candidates the detector rejected on its own are not in this pool, so the two filtering stages are kept apart: the detector's share is reported in Table 3 and the verifier's decisions in Table 4. This gives four counts, all in units of findings: a shipped candidate that matches is a TP and one that does not is an FP, while a candidate the verifier withholds is a false negative (FN) if it matches and a true negative (TN) if it does not. The conventional rate is then FPR $=$ FP/(FP + TN), reported in Table 4; its complement is the false-positive reduction proper.

TABLE 1. Availability of the proof-of-concept execution signal, decomposed into three states. Counts are pooled over all three detection models and five runs.

Benchmark	Emitted	Triggered	Compiled, not trig.	Failed to compile
DVD	693	92 (13.3%)	28 (4.0%)	573 (82.7%)
PyGoat	667	100 (15.0%)	559 (83.8%)	8 (1.2%)
NodeGoat	177	11 (6.2%)	164 (92.7%)	2 (1.1%)
SmartBugs	2994	0 (0.0%)	0 (0.0%)	2994 (100.0%)

A PoC that compiles but does not trigger the reported behaviour is an *informative negative* and is admitted to the verifier; a PoC that fails to compile carries no information and is withheld (Section III-D). Conflating the two is the failure mode we attribute to fuzzing in Section III-E. SmartBugs contracts use `^0.4.x` pragmas and ship without a buildable harness, so the signal is structurally unavailable there.

Any finding that does not correspond to a documented vulnerability in the project's standard is counted as FP, and is therefore treated as a negative in the confusion matrix above. Some findings marked as FP may be real vulnerabilities not covered by the benchmark. The reported FPR is consequently an upper bound and the reported precision a lower bound.

C. RECALL AND OPERATING POINTS

Mechanism-aware recall is a central definition in our evaluation. A file-level criterion—counting a vulnerability as caught whenever a shipped finding lands on the correct file—overstates recall, because a finding may hit the right file while misclassifying the vulnerability type. This is the same error our false-positive criterion already penalises as FP_t (Section IV-B); for consistency, we apply the same principle on the recall side. We therefore count a ground-truth vulnerability as caught only when a shipped finding both lands on the ground-truth file *and* reports a vulnerability class matching the ground-truth category. Recall retention is then the fraction of detector-caught true positives that survive verification.

Retention and end-to-end recall differ in their denominator. Retention is normalised by the number of vulnerabilities the detection stage surfaces, whereas end-to-end recall is normalised by the full set of labelled vulnerabilities in the benchmark. The gap between the two is the ceiling the detection stage imposes: on Damn Vulnerable DeFi, detection surfaces 5 of the 18 labelled vulnerabilities, so a retention of 28.0% corresponds to an end-to-end recall of only 7.8% (Table 5). We therefore report both throughout.

The two are not interconvertible. The confusion matrix in Table 4 is counted in units of findings, so the F1 score reported there is a finding-level quantity. Retention and end-to-end recall in Table 5 are counted in units of ground-truth vulnerabilities. A precision from the first table and a recall from the second do not combine.

This raises a practical question: how is a finding's vulnerability class determined? We resolve it primarily from the finding's CWE metadata, mapping the CWE to the ground-truth category; the full mapping is given in Table 15 so that it can be audited entry by entry. When no CWE is available we fall back to the L5 query type, and label the finding *unmappable* if neither source applies. We deliberately use only machine-generated sources: no keyword matching or manual assignment is applied, since either would introduce a labelling choice of our own into the criterion. The cost of that decision is a substantial unmappable fraction— 46.8% on Damn Vulnerable DeFi, 42.0% on PyGoat, 35.8% on NodeGoat, and 44.2% on SmartBugs (Table 16)—which we report rather than reduce. Mechanism-aware recall is consequently a lower bound on every benchmark, not only on NodeGoat.

TABLE 2. Reproducibility. Panel (a) decomposes the variance of the shipped-finding count into a detection and a verification component; panel (b) reports the inference settings of each stage.

(a) Variance decomposition

Model	Benchmark	SD total	SD det.	SD verif.	CV verif. (%)
gemma	DVD	5.22	0.00	5.29	15.1
	PyGoat	2.41	0.00	2.65	7.0
	NodeGoat	1.10	0.93	0.58	2.4
	SmartBugs	8.75	0.00	14.73	6.1
qwen	DVD	3.27	3.22	0.58	1.4
	PyGoat	11.39	4.94	10.26	21.1
	NodeGoat	3.08	2.92	1.00	7.7
	SmartBugs	5.93	2.05	5.57	3.5
Nemotron	DVD	1.34	1.21	0.58	3.0
	PyGoat	2.61	0.00	2.65	4.4
	NodeGoat	1.10	0.93	0.58	2.7
	SmartBugs	11.34	9.56	6.11	4.7

(b) Inference settings

Parameter	Value
Models	OpenRouter `qwen/qwen3-coder-next`, `google/gemma-4-31b-it`, `nvidia/nemotron-3-super-120b-a12b`
Temperature	0.2 (all verification; qwen and gemma detection); 0 (Nemotron detection)
top_p	0.7
top_k	not set
max_tokens	2048 (qwen, gemma); 16000 (Nemotron verification); 12000 (Nemotron detection)
Reasoning	Nemotron: on for verification, off for detection
Provider	Nemotron detection pinned (DeepInfra); others not pinned
Fallback	gemma and qwen detection fall back to Nemotron on a failed call
Seeds	1–5, passed to the serving API
Source hash	`pipeline.py` sha256:2a30e5c743a0b751 (round 1); c8d73b9457a11ff4 (rounds 2–4: adds only the RQ4 single-prompt path)
Python	3.12.3
Foundry	1.7.1
Backend	OpenRouter

In panel (a), total variance is measured across five independent full-pipeline runs and verification variance across three repeated verification runs on a single frozen candidate pool; the detection component is obtained by difference. Where it is reported as zero, the observed verification variance equals or exceeds the total, which at $n = 3$ and $n = 5$ lies within sampling error. In panel (b), the verification stage uses the same settings in every run; the Nemotron detection arm differs as listed. Seeds are passed to the API, but repeated passes with the same seed do not reproduce the same verdicts (RQ8), so replicate variation arises from sampling nondeterminism.

The two criteria are reported side by side, and the difference between them is itself a result. Under the mechanism-aware criterion, end-to-end recall falls from 53.3% to 7.8% on Damn Vulnerable DeFi and from 49.4% to 4.8% on SmartBugs, but does not change at all on NodeGoat. The drop is therefore not an artifact of the criterion itself, but of how well the CWE labels a finding carries align with the way the benchmark describes its vulnerabilities. NodeGoat's vulnerability types—SSRF, NoSQL injection, cross-site scripting—map cleanly onto CWE identifiers, whereas the semantic flaws in Damn Vulnerable DeFi, such as oracle manipulation and governance flash loans, have no close CWE counterpart. On SmartBugs the file-level figure is an upper bound rather than a measurement: every contract in the curated set is labelled vulnerable, so any finding that lands anywhere in the benchmark is credited as a true positive, and the resulting file-level precision of 95.1% reflects the labelling of the benchmark rather than the accuracy of the system. The degeneracy is visible in Table 4, where under the file-level criterion the true-negative count is zero and the false positive rate 100% at both operating points, invariant across all five runs: the only unlabelled candidates that reach the verifier are the eight that fall outside the labelled contracts, and they are shipped in every run.

Finally, our verification layer assigns each candidate one of three verdicts: confirmed, downgraded, or false positive. We report results at two operating points that share the same verdicts and differ only in the reporting threshold, not in any re-run. The *precision-first* point ships only findings of medium severity or above and discards downgraded findings, maximising the reduction in shipped findings. The *recall-first* point ships findings of low severity or above, retaining downgraded findings at low severity to maximise recall. The two points correspond to different deployment needs: precision-first suits settings where alert fatigue is the primary concern, such as CI/CD pipelines, while recall-first suits security audits where missing a vulnerability is costlier than reviewing an extra false positive.

D. RESEARCH QUESTIONS

RQ1: Can the verification stage reduce the number of false-positive findings?
RQ2: Can verification preserve recall, and what precision–recall trade-off does it offer?
RQ3: What does each signal contribute to verification?
RQ4: How does verification compare with a baseline and with prior work?
RQ5: Can the verification stage generalise across programming languages and vulnerability domains?
RQ6: What causes the variation across benchmarks?
RQ7a: How does detection model strength affect results?
RQ7b: Does the verification stage depend on the verification model?
RQ8: Are the verification stage's results stable across repeated runs?

E. RESULTS

RQ1 (Alert and False-Positive Reduction): Across all four benchmarks, verification consistently reduces the volume of shipped findings. We report results at two operating points, precision-first and recall-first. At the precision-first point, verification removes 64.6% of shipped findings on Damn Vulnerable DeFi, 63.3% on PyGoat, 62.2% on NodeGoat, and 54.6% on SmartBugs (Table 3). At the recall-first point, which retains downgraded findings at low severity, the reductions are smaller—48.1%, 33.6%, 17.8%, and 37.1% respectively—reflecting a precision–recall trade-off we examine in RQ2. At the precision-first point the reduction is consistent across benchmarks, spanning only ten percentage points from 54.6% to 64.6%; the larger source of variation is the detection model rather than the benchmark, which we examine in RQ7a. These figures count every withheld finding. Counted over unlabelled findings only, the false-positive reduction proper at the precision-first point is 63.3%, 61.8%, 49.3% and 52.7% (Table 6): within two points of the alert-count reduction on three benchmarks, and thirteen points below it on NodeGoat, where a larger share of the withheld alerts are labelled findings.

Reduction in volume, however, does not by itself establish that the retained findings are good ones. Table 4 therefore reports the full confusion matrix over the pool of candidates that reach the verifier, together with the conventional metrics it supports. At the precision-first point and under the mechanism-aware criterion, precision ranges from 3.6% on Damn Vulnerable DeFi to 25.8% on NodeGoat, finding-level recall from 19.3% to 31.2%, F1 from 6.0% to 24.2%, and the conventional false positive rate from 36.7% to 51.2%.

The low precision—3.6% on Damn Vulnerable DeFi—follows in part from the base rate of the candidate pool. Damn Vulnerable DeFi contributes 238 candidates against 18 labelled vulnerabilities, so even a perfect classifier could not reach a high precision there. Two further factors depress it. The conservative counting policy of Section IV-B charges any finding outside the documented list as a false positive, so the reported precision is a lower bound. And the mechanism-aware criterion is stricter than a file-level one: on Damn Vulnerable DeFi the same configuration attains a file-level precision of 42.3%, so most of the gap between 3.6% and that figure reflects class-matching rather than misplaced findings. A large reduction in volume and a low precision are therefore not in tension: the verification stage removes most of the shipped volume, but genuine vulnerabilities were rare in the pool it received.

The conventional false positive rate is the more informative measure here, because it is normalised by the negatives rather than by the positives and is consequently insensitive to the base rate. At 36.7% to 51.2%, it says that between one in three and one in two of the unlabelled candidates that reach the verifier are nevertheless shipped; equivalently, the verifier withholds about half to two thirds of them, which is the false-positive reduction proper of Table 6. That is the quantity

TABLE 3. Detection breadth, candidate volume, and alert-count reduction (Alert-cut) across all three detection models. Verification is Nemotron-3-Super-120B-A12B throughout. All values are mean ± standard deviation over five independent full-pipeline runs. Every column except Detect is a count of shipped findings and does not depend on the recall criterion.

				Detector self-rejection		Precision-first			Recall-first	
Detection model	Benchmark	Detect	Candidates	Baseline	Share (%)	Shipped	Alert-cut (%)	95% CI	Shipped	Alert-cut (%)
gemma-4-31B	DVD	4.0 ± 0.0	238 ± 2	92 ± 3	61.5 ± 1.3	38 ± 5	58.0 ± 7.0	[49.3, 66.7]	51 ± 6	43.9 ± 7.5
	PyGoat	10.0 ± 0.0	122 ± 0	105 ± 0	14.1 ± 0.4	39 ± 2	62.4 ± 2.2	[59.7, 65.1]	73 ± 2	30.5 ± 1.7
	NodeGoat	5.0 ± 0.0	56 ± 0	37 ± 1	34.3 ± 1.5	23 ± 1	38.0 ± 2.6	[34.8, 41.3]	26 ± 1	30.4 ± 2.3
	SmartBugs	16.6 ± 1.1	1051 ± 3	565 ± 5	46.2 ± 0.5	259 ± 9	54.2 ± 1.2	[52.7, 55.7]	363 ± 10	35.9 ± 1.4
qwen3-coder	DVD	5.0 ± 0.0	238 ± 2	111 ± 2	53.6 ± 0.6	39 ± 3	64.6 ± 2.7	[61.3, 67.9]	57 ± 2	48.1 ± 2.0
	PyGoat	10.0 ± 0.0	122 ± 1	111 ± 2	8.4 ± 1.5	41 ± 11	63.3 ± 10.2	[50.6, 76.1]	74 ± 6	33.6 ± 5.2
	NodeGoat	4.0 ± 0.0	56 ± 0	27 ± 1	52.1 ± 2.3	10 ± 3	62.2 ± 13.1	[46.0, 78.5]	22 ± 1	17.8 ± 2.4
	SmartBugs	21.2 ± 1.1	1010 ± 4	356 ± 5	64.7 ± 0.5	162 ± 6	54.6 ± 2.1	[52.0, 57.2]	224 ± 8	37.1 ± 2.4
Nemotron-120B	DVD	2.6 ± 0.9	214 ± 4	81 ± 6	62.0 ± 2.4	20 ± 1	75.8 ± 3.0	[72.1, 79.5]	32 ± 4	60.5 ± 4.2
	PyGoat	9.2 ± 0.8	115 ± 3	97 ± 5	15.8 ± 2.8	64 ± 3	33.7 ± 2.3	[30.8, 36.6]	70 ± 3	27.9 ± 3.0
	NodeGoat	4.8 ± 0.4	55 ± 1	38 ± 1	31.1 ± 1.4	22 ± 1	40.9 ± 3.9	[36.1, 45.7]	23 ± 1	38.8 ± 3.9
	SmartBugs	19.4 ± 2.8	929 ± 5	411 ± 5	55.8 ± 0.4	125 ± 11	69.6 ± 2.8	[66.2, 73.0]	174 ± 12	57.5 ± 3.0

Detect is the number of labelled vulnerabilities the detection stage surfaces under the mechanism-aware criterion, out of 18, 11, 8, and 143 for Damn Vulnerable DeFi, PyGoat, NodeGoat, and SmartBugs respectively. **Candidates** is the raw output of the detection stage. **Baseline** is the volume that survives the detector's own self-rejection, and **Share** is the fraction the detector removes on its own. Alert-cut is the alert-count reduction, computed against *Baseline*, so the reported reduction is attributable to the verification stage rather than to detector self-rejection; it counts every withheld finding, labelled or not, and the shares of unlabelled and of labelled findings withheld are given separately in Table 6. All three detection models use the inference settings in Table 2.

the verification stage controls, and it is what the alert-count reduction in Table 3 translates into at the level of individual decisions.

The more demanding comparison is precision before and after verification, which Table 6 reports for every detector. The reference for that comparison is removing the same number of alerts uniformly at random, which in expectation leaves precision at its pool value (the *none* column) and retains true positives in proportion to their share of the pool. In the main setting precision falls at the precision-first point on all four benchmarks: from 6.9% to 3.6% on Damn Vulnerable DeFi, from 22.4% to 19.1% on PyGoat, from 48.5% to 30.0% on NodeGoat, and from 10.2% to 6.3% on SmartBugs, because the stage withholds labelled findings at a higher rate (82%, 69%, 77% and 72%) than unlabelled ones (63%, 62%, 49% and 53%). Against the random reference the shortfall is systematic: the verifier retains fewer true positives than random removal would in four, four, five and five of the five runs, and the probability that random removal would retain no more than the observed number is 0.015, 0.092, 8×10^{-4} and 4×10^{-7} (hypergeometric, five runs pooled). Where the PoC verdict is absent or reliable the comparison reverses and the stage is selective: under gemma and Nemotron detection on NodeGoat it withholds 61% and 65% of the unlabelled findings but only 6% and 1% of the labelled ones, raising precision from 42.4% to 64.0% and from 37.8% to 63.1%, and there, as under Nemotron detection on PyGoat, the probability that random removal would retain as many true positives as the verifier does is below 10^{-4}. Two things account for the main-setting result. The PoC verdict removes true positives (RQ3), and on Damn Vulnerable DeFi and SmartBugs the labelled findings are so few—38 and 181 over five runs against 515 and 1601 unlabelled—that the class counted as false positive is dominated by findings the benchmark does not document, so the label cannot distinguish a verifier that removes hallucinations from one that removes at random. The recall-first point is less costly, and on PyGoat and NodeGoat it raises precision above that of the unverified pool (26.2% and 57.3%). The label-based measures remain our primary results because they cover every configuration and run and can be recomputed from the released data; to establish whether the withheld volume is hallucinated, we adjudicated a sample against the code.

We took every candidate that reached the verifier in one run (qwen3-coder detection, seed 1), stratified it by the verifier's decision at the precision-first point, sampled up to 40 per stratum (253 items), and had two LLM judges, neither of them a model used in the pipeline, classify each claim against the checked-out source as true, overstated or false, blind to the verifier's decision, the PoC state and the benchmark label (Table 7). Judge B provides the labels used below; judge A labelled the same items independently and agrees on whether a claim is true on 94% of them ($\kappa = 0.87$). Three results follow. First, the candidate pools differ in kind. On PyGoat and NodeGoat 71% and 84% of the candidates that reach the verifier are genuine weaknesses, most of them undocumented by the benchmark—CSRF-exempt state-changing endpoints, MD5 password hashing, cookies without the `Secure` flag—and only 3% and 0% are hallucinated, whereas on Damn Vulnerable DeFi and SmartBugs 26% and 43% are hallucinated and a further 61% and 44% overstated. The label-based precision of Table 4 is therefore a severe underestimate on the web benchmarks: of the shipped findings sampled on PyGoat, 80% are genuine against a label precision of 20% in the same sample, and on SmartBugs 20% against 8%. Second, the verifier is selective in the intended direction: pooled over the four benchmarks it withholds 71% of the hallucinated claims and 55% of the overstated ones against 43% of the genuine ones (66%, 59% and 44% under judge A). Third, the genuine findings it withholds are concentrated in the *downgraded* verdict: on PyGoat all 16 withheld findings that were downgraded are

TABLE 4. Finding-level classification quality of the verification stage under the mechanism-aware criterion and the file-level criterion, computed over the candidates that reach the verifier (the *Baseline* pool of Table 3), so that the detector's own self-rejection is not counted as a verification decision. Detection is qwen3-coder; verification is Nemotron-3-Super-120B-A12B. Entries are mean with standard deviation in parentheses, over five independent full-pipeline runs. All counts are in units of findings.

Benchmark	Thr.	TP	FP	FN	TN	Prec.	Rec.	F1	FPR
Mechanism-aware criterion									
DVD	P1	1.4(1.1)	37.8(3.3)	6.2(1.6)	65.2(3.2)	3.6(2.7)	19.3(16.6)	6.0(4.7)	36.7(3.0)
	R1	3.2(0.8)	54.2(1.8)	4.4(1.1)	48.8(3.4)	5.6(1.4)	42.5(12.3)	9.8(2.5)	52.7(2.4)
PyGoat	P1	7.8(2.5)	33.0(9.4)	17.2(2.5)	53.4(9.6)	19.1(4.0)	31.2(10.0)	23.4(5.2)	38.2(10.8)
	R1	19.4(0.9)	54.6(6.7)	5.6(0.9)	31.8(6.2)	26.4(2.8)	77.6(3.6)	39.4(3.5)	63.2(7.3)
NodeGoat	P1	3.0(2.5)	7.0(1.0)	10.0(2.5)	6.8(1.8)	25.8(19.2)	23.1(19.6)	24.2(19.3)	51.2(9.5)
	R1	12.6(0.5)	9.4(0.5)	0.4(0.5)	4.4(0.9)	57.3(1.9)	96.9(4.2)	72.0(2.5)	68.4(4.1)
SmartBugs	P1	10.2(2.3)	151.6(7.3)	26.0(2.1)	168.6(11.4)	6.3(1.5)	28.1(6.2)	10.3(2.4)	47.4(2.9)
	R1	16.6(2.7)	207.4(9.4)	19.6(2.2)	112.8(10.7)	7.4(1.3)	45.8(6.9)	12.8(2.2)	64.8(3.1)
File-level criterion									
DVD	P1	16.4(1.7)	22.8(4.7)	29.6(1.5)	41.8(3.5)	42.3(8.0)	35.6(3.4)	38.6(5.1)	35.2(6.6)
	R1	23.8(1.3)	33.6(2.5)	22.2(1.3)	31.0(3.0)	41.5(2.8)	51.7(2.7)	46.0(2.6)	52.0(3.9)
PyGoat	P1	17.2(4.8)	23.6(7.2)	32.6(5.5)	38.0(7.5)	42.2(5.4)	34.6(9.8)	37.6(7.1)	38.4(12.0)
	R1	34.2(2.5)	39.8(5.2)	15.6(1.7)	21.8(5.4)	46.3(3.2)	68.6(3.5)	55.3(2.9)	64.6(8.7)
NodeGoat	P1	3.0(2.5)	7.0(1.0)	10.0(2.5)	6.8(1.8)	25.8(19.2)	23.1(19.6)	24.2(19.3)	51.2(9.5)
	R1	12.6(0.5)	9.4(0.5)	0.4(0.5)	4.4(0.9)	57.3(1.9)	96.9(4.2)	72.0(2.5)	68.4(4.1)
SmartBugs	P1	153.8(5.9)	8.0(0.0)	194.6(9.6)	0.0(0.0)	95.1(0.2)	44.2(2.1)	60.3(2.0)	100.0(0.0)
	R1	216.0(8.3)	8.0(0.0)	132.4(9.1)	0.0(0.0)	96.4(0.1)	62.0(2.4)	75.5(1.8)	100.0(0.0)

P1 = precision-first threshold; R1 = recall-first. Precision, recall, F1 and FPR are percentages. The prediction is the verifier's ship/no-ship decision over the pool of candidates that reach it; the reference is whether the candidate matches a labelled vulnerability. Candidates the detector rejected on its own are excluded, so TP+FP+FN+TN equals the *Baseline* column of Table 3 and the pool of Table 6. FPR = FP/(FP+TN) is therefore the standard definition. Candidates matching no label are counted as negatives, so FPR is an upper bound and precision a lower bound. Percentages are computed per run and then averaged, so they need not equal the ratio of the averaged counts; F1 is averaged over all five runs, with F1 = 0 for a run without a true positive.

TABLE 5. Vulnerability-level recall under both matching criteria. Detection is qwen3-coder; verification is Nemotron-3-Super-120B-A12B. Entries are mean with standard deviation in parentheses, over five independent full-pipeline runs. All counts are in units of ground-truth vulnerabilities.

				Retention (%)		E2E recall (%)	
Benchmark	Criterion	GT	Detect	P1	R1	P1	R1
DVD	File	18	15.0	64.0(10.1)	81.3(8.7)	53.3(8.4)	67.8(7.2)
	Mechanism	18	5.0	28.0(22.8)	64.0(16.7)	7.8(6.3)	17.8(4.6)
PyGoat	File	11	11.0	92.7(4.1)	92.7(4.1)	92.7(4.1)	92.7(4.1)
	Mechanism	11	10.0	54.0(21.9)	88.0(4.5)	49.1(19.9)	80.0(4.1)
NodeGoat	File	8	4.0	35.0(28.5)	95.0(11.2)	17.5(14.3)	47.5(5.6)
	Mechanism	8	4.0	35.0(28.5)	95.0(11.2)	17.5(14.3)	47.5(5.6)
SmartBugs	File	143	94.2	75.0(3.2)	85.6(3.0)	49.4(1.5)	56.4(1.6)
	Mechanism	143	21.2	32.1(2.6)	44.3(4.7)	4.8(0.3)	6.6(0.8)

Retention and E2E recall are percentages. **GT** is the number of labelled vulnerabilities; **Detect** is the number the detection stage surfaces and is the denominator of retention, whereas E2E recall is normalised by GT. The gap between the two is the ceiling imposed by the detection stage. On SmartBugs every contract in the curated set is labelled vulnerable, which inflates the file-level rows.

genuine, and on NodeGoat 11 of the 13 withheld findings are, whereas of the findings withheld as *false positive* on the Solidity benchmarks, 29 of 32 on Damn Vulnerable DeFi and 21 of 24 on SmartBugs are overstated or hallucinated. The recall cost quantified in RQ3 is thus largely a cost of the precision-first threshold, which discards downgraded findings, rather than of wrong *false positive* verdicts.

RQ2 (Recall and Precision–Recall Trade-off): Verification does preserve recall, but the degree depends on the operating point (Table 5). At the precision-first point, retention is 28.0% on Damn Vulnerable DeFi, 54.0% on PyGoat, 35.0% on NodeGoat, and 32.1% on SmartBugs. At the recall-first point, retention rises to 64.0%, 88.0%, 95.0%, and 44.3% respectively. Retention alone, however, understates how much of each benchmark the pipeline actually finds: because it is normalised by what detection surfaced rather than by the full label set, the corresponding end-to-end recalls at the precision-first point are only 7.8%, 49.1%, 17.5%, and 4.8%. On Damn Vulnerable DeFi, for instance, detection surfaces 5 of the 18 labelled vulnerabilities, so a retention of 28.0% amounts to 1.4 vulnerabilities reported out of 18.

Both operating points are computed from the same findings and the same verification verdicts; they differ only in the report-layer ship threshold (Figure 2). The precision-first point drops all downgraded findings, so it cuts more false positives but retains less recall. The recall-first point ships downgraded findings at low severity, so it retains more recall but cuts fewer false positives. The two points serve different deployment needs: precision-first suits CI/CD pipelines, where alert fatigue is the primary concern, while recall-first

TABLE 6. Precision before and after verification under the mechanism-aware criterion, five runs pooled. **TP** and **FP** are the labelled and unlabelled findings in the unverified pool per run; **Prec.** is the precision of the pool (none), of the precision-first point (P1) and of the recall-first point (R1); **Removed** is the percentage of the pool's labelled (TP) and unlabelled (FP) findings that verification withholds at each point. Verification is Nemotron-3-Super-120B-A12B throughout.

		Pool		Prec. (%)			Removed, P1 (%)		Removed, R1 (%)	
Detector	**Benchmark**	**TP**	**FP**	**none**	**P1**	**R1**	**TP**	**FP**	**TP**	**FP**
gemma	DVD	6.8	85.0	7.4	5.7	6.2	67.6	57.4	52.9	43.3
	PyGoat	24.0	80.8	22.9	19.8	26.6	67.5	60.9	19.2	33.9
	NodeGoat	15.6	21.2	42.4	64.0	60.9	6.4	61.3	0.0	52.8
	SmartBugs	39.6	525.8	7.0	4.3	5.6	71.7	52.9	48.5	34.9
qwen	DVD	7.6	103.0	6.9	3.6	5.6	81.6	63.3	57.9	47.4
	PyGoat	25.0	86.4	22.4	19.1	26.2	68.8	61.8	22.4	36.8
	NodeGoat	13.0	13.8	48.5	30.0	57.3	76.9	49.3	3.1	31.9
	SmartBugs	36.2	320.2	10.2	6.3	7.4	71.8	52.7	54.1	35.2
Nemotron	DVD	4.4	77.0	5.4	1.0	5.0	95.5	74.8	63.6	60.3
	PyGoat	21.6	75.6	22.2	27.6	27.1	17.6	38.4	12.0	32.5
	NodeGoat	14.2	23.4	37.8	63.1	61.7	1.4	65.0	0.0	62.4
	SmartBugs	33.8	377.0	8.2	5.9	7.1	78.1	68.9	63.3	57.0

A verifier that is selective against the benchmark labels removes unlabelled findings at a higher rate than labelled ones and raises precision. Removing the same number of alerts uniformly at random would leave precision at the value in column *none* in expectation, so that column is also the random-removal reference for P1 and R1. FP includes findings the benchmark does not document (Section IV-B).

TABLE 7. Adjudication of verifier decisions against the source code (qwen3-coder detection, seed 1, precision-first threshold). Candidates that reached the verifier were stratified by its decision and up to 40 per stratum were sampled (all of them where fewer exist). Two LLM judges, blind to the verifier's decision, the PoC state and the benchmark label, classified each claim as true (a genuine weakness, documented by the benchmark or not), overstated (the code pattern exists but the claimed exploitability does not hold) or false (the claim misdescribes the code). Panel (a) gives the composition of each stratum under judge B and the share of items on which the two judges agree; panel (b) gives, for each class, the percentage of such candidates that the verifier withheld, weighting strata by their size, under judge B, judge A and the agreed subset (C).

(a) Composition of each stratum (%, judge B)

Benchmark	**Stratum**	**Pool**	n	TRUE	OVERST.	FALSE	**Agree**
DVD	shipped	34	34	15	74	12	85
	withheld	70	40	12	55	32	70
PyGoat	shipped	56	40	80	18	2	78
	withheld	50	40	60	38	2	88
NodeGoat	shipped	6	6	83	17	0	83
	withheld	13	13	85	15	0	100
SmartBugs	shipped	154	40	20	50	30	80
	withheld	199	40	8	40	52	72

(b) Share of each class withheld by the verifier (%)

	TRUE			OVERSTATED			FALSE		
Benchmark	B	A	C	B	A	C	B	A	C
DVD	64	69	57	61	57	56	85	80	81
PyGoat	40	45	43	66	53	75	47	47	100
NodeGoat	69	65	69	67	100	100	—	—	—
SmartBugs	33	24	27	51	59	53	69	63	66
All	43	44	43	55	59	56	71	66	69

253 items. Judge B is Claude Fable 5.1 and judge A Claude Opus 4.8, neither of which is used in the pipeline; they agree on whether a claim is true on 94% of items ($\kappa = 0.87$) and on the five-way label with $\kappa = 0.70$. **Pool** is the number of candidates in the stratum. NodeGoat has no FALSE claims in the sample, so that cell is undefined. The items, the judges' justifications and the protocol are in the replication package.

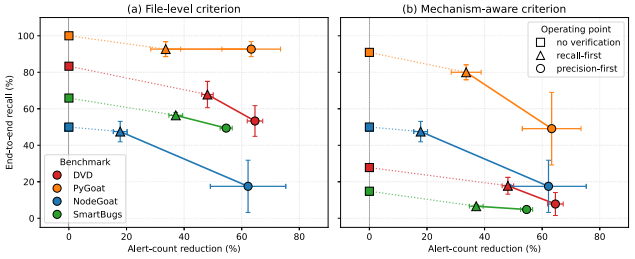


FIGURE 2. Precision–recall trade-off across the four benchmarks under two recall criteria. Each benchmark is shown at three operating points: no verification (squares), the recall-first reporting threshold (triangles), and the precision-first threshold (circles). All three share the same verification verdicts and differ only in the reporting threshold. Error bars denote one standard deviation over five independent full-pipeline runs. Panel (a) credits a ground-truth vulnerability when a shipped finding lands on its file; panel (b) additionally requires a matching vulnerability class. The vertical position of the square marker is the recall ceiling imposed by the detection stage.

suits security audits, where missing a vulnerability is costlier than reviewing an extra false positive.

Comparing the two extremes reveals what the downgraded verdict is worth, and it is not what recall retention alone suggests. On SmartBugs, retention rises only 12 percentage points, yet the absolute gain is 2.6 vulnerabilities—more than the 2.4 that NodeGoat gains with a 60-point rise. The difference is a denominator effect: detection surfaces 21.2 labelled vulnerabilities on SmartBugs against 4.0 on NodeGoat. What separates the two is the price of that gain. NodeGoat buys its 2.4 vulnerabilities with 2.4 additional false positives, whereas SmartBugs pays 55.8 for 2.6 (Table 4)—a ratio of 1.000 against 0.046, a factor of about 22.

The ordering is consistent with the size of the candidate pool rather than with detection quality. NodeGoat receives 27 candidates, PyGoat and Damn Vulnerable DeFi 111 each, and SmartBugs 356, and their ratios fall monotonically at 1.000, 0.157, 0.110, and 0.046. Detection quality does not order them the same way: NodeGoat surfaces only 4 of its 8 labelled vulnerabilities yet gives the best return,

TABLE 8. Average marginal contribution of each verification signal, computed over the complete power set of the four signals ($2^4 - 1 = 15$ non-empty subsets), qwen3-coder detection, precision-first threshold. Each entry is the mean change in shipped-finding count when the signal is added to a subset that does not already contain it. The full power set is given in Table 12.

Signal	DVD	PyGoat	NodeGoat	SmartBugs
Identity	+0.3	+3.0	+0.1	−1.3
Source	+5.7	+25.8	+7.5	+13.1
Concern	+20.3	+1.0	+0.2	+82.9
PoC	+2.0	−18.2	−7.9	—

A positive value means adding the signal increases the number of shipped findings; a negative value means it suppresses them. Seven subset pairs per signal on the first three benchmarks; three on SmartBugs, where subsets containing the PoC signal are structurally unavailable (Table 1).

while PyGoat surfaces 10 of 11 and gives the second. With four benchmarks we can only note the agreement, not establish the mechanism, but the practical implication is stable: on large candidate pools the downgraded verdict is expensive, and the precision-first point is the better default.

The standard deviations in Table 5 are large on NodeGoat—28.5 percentage points on retention at the precision-first point—because the benchmark contains only 8 labelled vulnerabilities and the detection stage surfaces 4 of them, so a single vulnerability moves retention by 25 points. The NodeGoat figures should be read accordingly.

RQ3 (Signal Contribution): We propose a multi-signal approach to reduce false positives, but does each signal truly contribute? To examine this we ablate the four signals over their complete power set, running all $2^4 - 1 = 15$ non-empty subsets (Table 12). We report the average marginal contribution of each signal rather than a leave-one-out difference, because the latter measures the effect of removing a signal at a single point—the full configuration—whereas the former averages the change over the seven subset pairs in which the signal is the only difference (Table 8). Statistical comparisons for every configuration are given in Table 13.

The identity signal turns out to be redundant. Adding it changes the number of shipped findings by +0.3 on Damn Vulnerable DeFi, +3.0 on PyGoat, +0.1 on NodeGoat, and −1.3 on SmartBugs—negligible against the +25.8 that source contributes on PyGoat or the +82.9 that concern contributes on SmartBugs. Removing identity is statistically significant in only 3 of the 12 model-benchmark cells. The clearest case is SmartBugs under gemma detection, where 419 findings are shipped under the full configuration but not without identity, and 424 the other way round, out of 843 findings on which the two configurations disagree ($p = 1.00$ after Holm correction). With 843 discordant pairs and a split that is almost exactly even, this is a well-powered null rather than an inconclusive test. A plausible explanation is that the source code already carries the information identity would supply, since function declarations appear in the code the verifier is given.

Source and concern divide the labour between them, and the division follows the programming language. On the two Solidity benchmarks, concern dominates: adding it ships 20.3 more findings on Damn Vulnerable DeFi and 82.9 more on SmartBugs, against 5.7 and 13.1 for source. On Python and JavaScript the ordering reverses: source contributes 25.8 on PyGoat and 7.5 on NodeGoat, while concern contributes only 1.0 and 0.2. The effect of removing source is most pronounced under the stronger Nemotron model, where PyGoat's false positives fall from 46.6 to 24.0 and NodeGoat's true positives fall from 14.0 to 0.8—that is, the verifier confirms almost nothing without it. Removing source is significant in 10 of the 12 cells and removing concern in 6, both far above the 3 for identity. The pattern is consistent with the kind of flaw each benchmark contains: the vulnerabilities in Damn Vulnerable DeFi and SmartBugs are semantic—oracle manipulation, governance flash loans, rounding errors—and are not visible in a single function without a statement of what to look for, whereas SSRF, cross-site scripting, and injection flaws in PyGoat and NodeGoat are apparent in the code itself.

Removing the PoC verdict has the opposite effect: shipped findings rise, because a verdict that compiled but did not trigger the reported behaviour no longer counts against a finding. Its average marginal contribution is accordingly negative (−18.2 on PyGoat and −7.9 on NodeGoat)—that is, the signal suppresses shipped findings rather than confirming them. The magnitudes track the availability of informative counterevidence in Table 1: 83.8% of the PoCs emitted on PyGoat and 92.7% of those on NodeGoat compiled without triggering, and those two benchmarks show the strongest suppression. Damn Vulnerable DeFi is the exception at +2.0, close to zero, because 82.7% of its PoCs fail to compile and the verdict is therefore withheld rather than admitted; on SmartBugs synthesis was attempted for 2994 candidates and none compiled, so the signal is absent entirely.

The verdict is not free, however. At the precision-first point it removes true positives faster than false positives: under qwen detection, adding the PoC verdict to the other three signals lowers PyGoat's true positives from 17.4 to 7.8 and its false positives from 47.6 to 33.0, and NodeGoat's from 13.0 to 3.0 against 8.4 to 7.0 (Table 12). Precision therefore falls rather than rises, from 26.8% to 19.1% on PyGoat and from 60.7% to 30.0% on NodeGoat. The cost is confined to the precision-first point: at the recall-first point 94–100% of the true positives survive, because the verdict moves genuine findings to *downgraded* rather than to *false positive*. It is also confined to the detectors whose PoCs actually run. PyGoat shows the same pattern under gemma detection (true positives 20.6 to 7.8 against false positives 50.0 to 31.6, Table 9), whereas the Nemotron arm emits almost no PoCs on PyGoat and NodeGoat, so its full and no-PoC configurations differ only by run-to-run variation, and on Damn Vulnerable DeFi, where 83% of the PoCs fail to compile and the verdict is withheld, neither count moves by more than a fifth. The reason lies in what the PoC exercises. On the Python and JavaScript benchmarks the synthesised PoC is a standalone script that re-implements the flagged function

TABLE 9. Leave-one-out signal ablation across all three detection models (precision-first threshold, mechanism-aware criterion). Entries are mean with standard deviation in parentheses, over five independent full-pipeline runs. Configurations within a run share the same candidate pool and are therefore paired; statistical comparisons are given in Table 13.

		Full		−Identity		−Concern		−Source		−PoC	
Model	**Benchmark**	**TP**	**FP**	**TP**	**FP**	**TP**	**FP**	**TP**	**FP**	**TP**	**FP**
gemma	DVD	2.2(0.4)	36.2(4.9)	3.2(1.5)	32.8(5.2)	0.8(0.4)	19.8(4.2)	2.0(0.7)	27.0(7.2)	2.8(1.3)	35.2(4.0)
	PyGoat	7.8(1.6)	31.6(2.1)	5.2(1.9)	25.0(2.4)	5.8(2.7)	31.2(3.1)	0.0(0.0)	10.2(2.4)	20.6(0.5)	50.0(1.6)
	NodeGoat	14.6(0.9)	8.2(1.3)	14.0(1.6)	7.4(0.5)	14.4(1.1)	7.6(1.3)	0.0(0.0)	6.4(0.9)	15.6(0.5)	9.0(0.0)
	SmartBugs	11.2(3.0)	247.8(6.7)	12.2(0.8)	247.8(12.4)	7.4(2.4)	208.2(26.0)	12.8(2.8)	234.8(11.9)	—	—
qwen	DVD	1.4(1.1)	37.8(3.3)	2.4(0.9)	38.0(7.3)	1.4(1.1)	23.4(2.9)	2.2(0.8)	36.2(4.5)	1.6(0.5)	32.2(3.5)
	PyGoat	7.8(2.5)	33.0(9.4)	7.0(1.0)	23.8(5.5)	8.4(1.9)	35.0(7.9)	3.6(1.3)	21.2(5.8)	17.4(0.9)	47.6(2.3)
	NodeGoat	3.0(2.5)	7.0(1.0)	2.0(1.6)	6.6(0.9)	3.0(3.0)	6.6(0.5)	0.2(0.4)	5.8(0.4)	13.0(0.0)	8.4(0.9)
	SmartBugs	10.2(2.3)	151.6(7.3)	11.4(1.8)	155.4(4.0)	7.8(1.5)	107.6(4.7)	15.8(1.6)	171.2(6.8)	—	—
Nemotron	DVD	0.2(0.4)	19.4(1.5)	0.4(0.5)	19.6(3.4)	0.2(0.4)	13.6(2.9)	0.4(0.5)	12.6(3.2)	0.2(0.4)	19.4(1.1)
	PyGoat	17.8(2.6)	46.6(1.1)	17.6(1.5)	41.8(1.6)	18.2(1.5)	48.6(3.3)	7.0(0.7)	24.0(2.5)	18.0(1.2)	46.4(2.4)
	NodeGoat	14.0(0.7)	8.2(0.8)	14.0(0.7)	7.8(0.4)	14.2(0.4)	8.4(0.9)	0.8(0.8)	6.6(0.5)	14.0(0.7)	7.8(0.4)
	SmartBugs	7.4(1.7)	117.4(11.2)	6.0(2.1)	124.0(7.6)	5.2(2.3)	97.6(4.3)	8.6(2.5)	94.4(6.7)	—	—

−PoC is omitted for SmartBugs, where the PoC signal is structurally unavailable and the configuration is identical to Full. TP and FP are counts of shipped findings under the mechanism-aware criterion.

with mocked dependencies rather than a request against the running application, so a PoC that runs without reaching the reported state says more about the simulation than about the code; the verifier nonetheless follows the instruction to weigh the verdict heavily, and downgrades the finding. The claim the data support is therefore that the PoC verdict suppresses shipped findings, at a recall cost that depends on whether the PoC exercises the real target.

Two of the 48 model-benchmark-ablation cells contradict the expected direction. For identity, concern, and source the expected effect of removal is a decrease in shipped findings, since the verifier loses evidence it needs to confirm; for the PoC verdict it is an increase, since it loses counterevidence. Under qwen detection, removing the PoC verdict on Damn Vulnerable DeFi ships fewer findings rather than more (69 shipped only with the signal against 42 only without it, $p = 0.040$), and removing source on SmartBugs ships more rather than fewer (248 against 374, $p = 1.5 \times 10^{-6}$). We report these explicitly: the claim supported by the data is not that every signal improves the full configuration everywhere.

What the ablation does support is narrower and more specific. Three of the four signals carry measurable weight, and they act in different ways: source and concern supply the evidence on which a finding is confirmed, with their relative importance set by the language, while the PoC verdict supplies counterevidence that suppresses shipped findings, at the recall cost described above. Identity shows no measurable contribution. The three that matter are non-redundant—each carries evidence the others do not—but they are not statistically independent, as the pairwise agreement in Table 14 shows.

RQ4 (Comparison with Baseline and Prior Work): We compare verification against two references: a no-verification baseline and the false-positive rates reported in prior work. Against the baseline, where detection findings are shipped without verification, the verification layer removes a large fraction of the shipped volume: 72 findings on Damn Vulnerable DeFi, 70 on PyGoat, 17 on NodeGoat, and 194 on SmartBugs (Table 3). Because the baseline already excludes the candidates the detector rejects on its own—between 8.4% and 64.7% of the raw output, depending on the configuration (Section IV-B)—this reduction is attributable to the verification layer rather than to detector self-rejection. Whether the multi-signal design is superior to a naive single-signal check is answered in RQ3: three of the four signals carry measurable weight and act in different ways, so no single signal is sufficient on its own.

Against prior work, recent studies report false-positive rates exceeding 50% for LLM-based vulnerability detection [1], [35], which is the scale of problem our verification stage addresses. Those figures come from different benchmarks and pipelines, so we report them as context rather than as a controlled comparison.

RQ5 (Cross-Language Generalisation): Verification generalises across languages and domains. Across our four benchmarks spanning three languages—Solidity (Damn Vulnerable DeFi, SmartBugs), Python (PyGoat), and JavaScript (NodeGoat)—verification reduces the shipped volume in every case, by 54.6% to 64.6% (Table 3). More broadly, every one of the 36 configurations we evaluate—three detection models, three verification models, and four benchmarks—yields a positive reduction, spanning 16.3% to 81.8%. This generality follows from the form of the signals rather than from their content: asking which file and function a concern refers to, what the surrounding source code says, what the concern claims, and whether an execution attempt bore it out are questions that arise in any language, and none of them needs to be redesigned per target. Which signal carries the work does change with the language, as RQ3 shows, but the set of questions does not. Our approach is therefore more general than prior work confined to a single domain, such as memory-safety vulnerabilities [5] or Web3 price manipulation [6].

Across benchmarks the reduction is consistent, spanning ten percentage points from 54.6% on SmartBugs to 64.6% on Damn Vulnerable DeFi. Recall retention is not: it ranges from 28.0% to 54.0% at the same operating point. We examine what drives the variation in recall in RQ6.

RQ6 (Cause of Variation): We have seen that the effect varies across benchmarks, but what causes this variation—the programming language or the vulnerability domain? The answer is neither: RQ5 already showed that the verification layer works across languages. It is also worth being precise about what varies. At the precision-first point the alert-count reduction spans only ten percentage points across the four benchmarks, from 54.6% to 64.6%, whereas end-to-end recall spans 4.8% to 49.1%, an order of magnitude. Three factors are measurable in our data: detection breadth, the availability of the PoC verdict, and the capability of the verifier.

First, detection breadth sets the ceiling on recall. Detection surfaces 10.0 of the 11 labelled vulnerabilities on PyGoat, 4.0 of 8 on NodeGoat, 5.0 of 18 on Damn Vulnerable DeFi, and 21.2 of 143 on SmartBugs, capping end-to-end recall at 90.9%, 50.0%, 27.8%, and 14.8% respectively no matter how well verification performs. Detection also frequently misclassifies the vulnerability mechanism—reporting, for example, an arbitrary-call bug as reentrancy—and the size of that effect is visible in the gap between the two recall criteria: on Damn Vulnerable DeFi, end-to-end recall falls from 53.3% at the file level to 7.8% once a matching vulnerability class is required (Section IV-C). Most of what detection surfaces there lands on the right contract but describes the wrong mechanism.

Second, the availability of the PoC verdict varies by almost the full range. Of the PoCs emitted, 98.9% on NodeGoat and 98.8% on PyGoat carry information, against 17.3% on Damn Vulnerable DeFi and none at all on SmartBugs, whose 2018-era contracts ship without a buildable harness (Table 1). Availability is set by the detection arm that synthesises the PoC: the Nemotron arm, for instance, emits 5 PoCs across the 552 PyGoat candidates that reach deep analysis and 2 across 243 on NodeGoat, none of which compile. On PyGoat and NodeGoat a verdict is almost always present, but it is a verdict on a re-implementation of the flagged function rather than on the running application, which is why suppression is strongest on exactly these two benchmarks and why it reaches true positives as well (RQ3).

Third, the verifier itself contributes. Holding the candidate pool fixed and varying only the verification model moves the benchmark-averaged reduction by 7 to 27 percentage points, with the same ordering on three independently produced pools (Table 10); we examine this in RQ7b.

Detection—particularly for subtle semantic vulnerabilities such as those in DeFi—is therefore the true bottleneck for end-to-end recall, since no verification stage can report a vulnerability the detector never surfaced. The variation in the reduction itself, however, is not attributable to detection alone: it reflects all three factors, two of which lie inside the verification stage.

RQ7a (Effect of Detection Model Strength): We vary the detection model across gemma, qwen, and Nemotron, holding the verification model fixed to Nemotron (Table 3). The detection model strongly affects how many candidates reach verification. Under the mechanism-aware criterion, qwen surfaces the most labelled vulnerabilities on three of the four benchmarks (tied with gemma on PyGoat)—21.2 of 143 on SmartBugs and 5.0 of 18 on Damn Vulnerable DeFi—while Nemotron is the most conservative, surfacing only 2.6 of 18 on Damn Vulnerable DeFi. Notably, the strongest model does not detect the most: used as a detector, Nemotron reports the fewest candidates.

The reduction that verification achieves downstream also depends on the detection model, but not uniformly. Across benchmarks it spans 38.0–62.4% with gemma, 54.6–64.6% with qwen, and 33.7–75.8% with Nemotron; the confidence intervals do not overlap in most cells. Nemotron produces both the highest reduction, 75.8% on Damn Vulnerable DeFi, and the lowest, 33.7% on PyGoat, so no model dominates. The Damn Vulnerable DeFi figure has a mechanical explanation: Nemotron surfaces only 2.6 of the 18 labelled vulnerabilities there, so the candidates that survive its own filtering are comparatively clean and a larger fraction of the remaining volume can be removed—at the cost of a lower recall ceiling. The same reasoning does not hold on PyGoat, where Nemotron surfaces 9.2 of 11 yet achieves the lowest reduction of the three, so detector conservatism alone does not determine downstream reduction. Because this experiment varies the detection model, it cannot say whether the verification stage itself depends on which model performs verification; we address that in RQ7b.

RQ7b (Dependence on the Verification Model): RQ7a varies the detection model, so the candidate pools it compares differ in size—81 versus 111 candidates on Damn Vulnerable DeFi, for example—and any difference cannot be attributed to the verification stage. To isolate that stage we freeze three independently produced candidate pools, one from each detection model, and vary only the verification model (Table 10). Cells in which the verifier is the same model that produced the pool are self-verifying and are marked separately.

The architecture transfers: every one of the 36 configurations we evaluate yields a positive reduction, spanning 16.3% to 81.8%. What differs is the magnitude. Averaged over the four benchmarks, Nemotron reaches 55.2% on the gemma pool, 61.0% on the qwen pool and 55.3% on its own pool, gemma 39.5%, 39.3% and 54.0%, and qwen 36.8%, 33.6% and 48.1%—a spread of 7 to 27 percentage points, with confidence intervals that do not overlap in most cells. The ranking is identical on all three pools and Nemotron is best in ten of the twelve pool–benchmark cells, so it is not an artifact of which model produced the candidates. Self-verification accounts for little of the spread: gemma's reduction on its own pool differs from its reduction on the qwen pool

by +0.2 percentage points, and qwen's by −3.2, an order of magnitude below the difference between verifiers on those pools.

Two conclusions follow. The verification stage is not model-agnostic—the choice of verifier moves the reduction by as much as the choice of benchmark does—so the variation in the overall result cannot be attributed to the detection stage alone. The stronger model, however, is not simply the better verifier. Volume is only half of the comparison, and the precision and retention columns of Table 10 reverse the ranking: averaged over the twelve pool–benchmark cells, Nemotron retains 52.6% of the vulnerabilities the detector surfaced against 74.4% for gemma and 72.3% for qwen, at a precision of 20.6% against 25.4% and 25.6%. On the gemma and qwen PyGoat pools the gemma verifier retains every detector-surfaced vulnerability at a precision of about 27%, whereas Nemotron retains 51% to 52% at 16% to 19%: the additional volume it removes contains true positives at a higher rate than the volume gemma removes. The choice of verifier is therefore a precision–recall trade-off rather than a ranking. The reasoning model removes the most volume; the smaller, non-reasoning models remove less but lose almost no true positives. The division of labour in Section IV-A—detection runs cheaply at scale, verification applies the stronger model—is supported for volume reduction but not for precision, and a deployment that values recall should prefer the gemma verifier at the precision-first point or Nemotron at the recall-first point.

RQ8 (Are the verification stage's results stable across repeated runs?): Reporting that a total count is stable is not enough, since two runs can ship the same number of findings while disagreeing on which ones. We therefore isolate the variability of the verification stage from that of detection and report the two components separately. Total variance is measured across the five independent full-pipeline runs, in which both stages are re-executed. Verification variance is measured separately, across three repeated verification passes over a single frozen candidate pool, so that only the verification stage varies. Since the two stages contribute independently, the detection component follows by difference, $\mathrm{SD}_{\mathrm{det}} = \sqrt{\mathrm{SD}^2_{\mathrm{total}} - \mathrm{SD}^2_{\mathrm{verif}}}$ (Table 2). The coefficient of variation of the verification stage ranges from 1.4% to 21.1% across the twelve model-benchmark cells, a spread wide enough to be worth decomposing.

The decomposition shows that neither stage dominates in general. On SmartBugs under Nemotron detection the detection component is 9.56 against 6.11 for verification, so most of the run-to-run variation originates upstream. On the same benchmark under gemma detection the ordering reverses: the detection component is 0.00 against 14.73 for verification. The source of variability is therefore a property of the configuration rather than of the pipeline. A consistency check supports the decomposition: the largest verification coefficient of variation, 21.1% on PyGoat under qwen detection, corresponds to the widest confidence interval on the alert-count reduction in Table 3, 63.3% ± 10.2%.

Four cells report a detection component of exactly zero: Damn Vulnerable DeFi, PyGoat, and SmartBugs under gemma detection, and PyGoat under Nemotron. In these the measured verification variance equals or exceeds the measured total, so the difference under the square root is non-positive and we report zero. This does not mean the detection stage is deterministic there; it means the two estimates differ by less than the sampling error of $n = 3$ and $n = 5$ measurements, and the detection component cannot be resolved from the verification component at that sample size.

Counts, however, can be stable while the findings behind them change, so we also measure agreement at the level of individual candidates. Each frozen pool is verified three times, and Table 11 reports the share of candidates on which all three passes make the same ship decision at the precision-first point (unanimity) and Fleiss' κ over the three passes. With Nemotron as the verifier, unanimity across the twelve seed-1 pools ranges from 48% to 97% and κ from 0.31 to 0.97; on the qwen PyGoat pool, whose three passes ship 60, 46 and 40 findings, only 69% of the 112 candidates receive the same decision every time. With gemma or qwen as the verifier, the sixteen cross-verifier cells show unanimity of 94% to 100% and κ of 0.90 to 1.00. The most aggressive verifier is thus also the least stable, the same trade-off that RQ7b reports. The same passes settle the question of seeds: all three use the same seed value, and two passes still differ on up to 123 candidates (SmartBugs under gemma detection), so the seed is not honoured by the serving API, the variance we report is sampling variance, and fixing the seed would not remove it. A deployment that requires reproducible verdicts should therefore use a non-reasoning verifier or aggregate several passes.

The answer to this question is therefore conditional. Whether the pipeline is stable, and which stage is responsible when it is not, depends on the configuration and, for the verifier, on the model; Table 2 makes the count-level attribution available per cell and Table 11 the finding-level one. What the decomposition rules out is the simpler claim that the variation can be assigned to the detection stage alone, which is consistent with the three factors identified in RQ6.

V. CASE STUDIES

To illustrate how the four signals interact in practice, we walk through three representative cases: a hallucination correctly rejected, a finding correctly downgraded, and a case the verifier judged incorrectly.

A. CORRECTLY REJECTING A HALLUCINATION

On Damn Vulnerable DeFi, the detection stage misclassified an arbitrary-call bug as reentrancy. Among the four signals, the concern claimed reentrancy, but the source code showed that the function is guarded by `nonReentrant` and

TABLE 10. Controlled cross-model comparison of the verification stage. Three candidate pools are frozen (gemma-4-31B, qwen3-coder and Nemotron detection, seeds 1–3) and each is adjudicated by all three verifiers, so verifier identity is the only variable. Diagonal cells, where the verifier is the same model that produced the pool, are marked *self*. Entries are mean with standard deviation in parentheses.

Pool	Bench.	Verifier	Rel.	Pool	Alert-cut	95% CI	Prec.	Ret.
gemma	DVD	Nemotron	cross	91(4)	59.7(7.6)	53.9–65.5	7.5(1.8)	63.9(18.2)
		gemma	self	91(4)	58.1(1.7)	56.7–59.4	9.7(2.7)	77.8(19.5)
		qwen	cross	91(4)	47.7(2.6)	45.7–49.7	6.2(1.5)	58.3(12.5)
	PyGoat	Nemotron	cross	105(1)	65.8(3.9)	62.8–68.8	15.8(4.7)	52.2(16.4)
		gemma	self	105(1)	24.7(0.9)	24.1–25.4	26.4(0.5)	100.0(0.0)
		qwen	cross	105(1)	34.5(1.5)	33.3–35.7	26.8(1.1)	86.7(5.0)
	NodeGoat	Nemotron	cross	37(1)	39.7(4.2)	36.4–42.9	62.0(4.5)	80.0(10.0)
		gemma	self	37(1)	33.3(0.8)	32.7–33.9	63.5(0.8)	100.0(0.0)
		qwen	cross	37(1)	35.7(2.0)	34.2–37.2	63.5(2.0)	88.9(10.5)
	SmartBugs	Nemotron	cross	568(5)	55.7(2.3)	53.9–57.5	4.3(1.0)	48.3(7.2)
		gemma	self	568(5)	41.8(0.8)	41.1–42.4	4.4(0.2)	71.7(3.8)
		qwen	cross	568(5)	29.3(0.9)	28.6–30.0	6.3(0.5)	77.5(8.4)
qwen	DVD	Nemotron	cross	112(0)	64.2(2.3)	62.4–66.0	4.7(1.9)	37.8(15.6)
		gemma	cross	112(0)	62.5(3.0)	60.2–64.8	9.9(1.4)	66.7(10.0)
		qwen	self	112(0)	44.7(3.8)	41.8–47.7	7.7(1.2)	75.6(16.7)
	PyGoat	Nemotron	cross	113(1)	67.9(10.1)	60.1–75.7	19.1(5.9)	51.1(22.6)
		gemma	cross	113(1)	31.0(2.3)	29.2–32.7	26.9(0.8)	100.0(0.0)
		qwen	self	113(1)	39.3(5.7)	35.0–43.7	29.1(1.9)	93.3(5.0)
	NodeGoat	Nemotron	cross	26(2)	56.1(9.8)	48.6–63.7	32.3(11.8)	47.2(23.2)
		gemma	cross	26(2)	16.3(4.1)	13.1–19.5	59.1(0.0)	100.0(0.0)
		qwen	self	26(2)	27.7(3.9)	24.7–30.7	59.7(3.9)	72.2(8.3)
	SmartBugs	Nemotron	cross	357(6)	55.9(2.5)	54.0–57.9	6.6(1.1)	31.8(1.9)
		gemma	cross	357(6)	47.4(1.2)	46.5–48.3	6.9(0.4)	37.4(2.3)
		qwen	self	357(6)	22.7(1.1)	21.8–23.5	8.9(0.5)	55.3(2.2)
Nemotron	DVD	Nemotron	self	81(6)	76.9(1.2)	75.9–77.8	0.6(1.8)	3.7(11.1)
		gemma	cross	81(6)	81.8(2.5)	79.9–83.7	2.4(3.7)	11.1(16.7)
		qwen	cross	81(6)	68.7(2.9)	66.5–71.0	3.1(2.6)	25.9(22.2)
	PyGoat	Nemotron	self	94(1)	33.8(2.9)	31.6–36.0	25.9(1.4)	88.3(12.7)
		gemma	cross	94(1)	26.0(2.0)	24.5–27.5	26.4(2.1)	96.7(5.0)
		qwen	cross	94(1)	28.6(2.3)	26.8–30.4	27.3(2.2)	96.7(5.0)
	NodeGoat	Nemotron	self	38(1)	40.8(2.9)	38.6–43.1	61.8(2.8)	93.3(10.0)
		gemma	cross	38(1)	40.8(2.1)	39.3–42.4	61.7(2.7)	93.3(10.0)
		qwen	cross	38(1)	41.1(2.6)	39.1–43.1	62.0(2.5)	93.3(10.0)
	SmartBugs	Nemotron	self	413(3)	69.8(2.0)	68.3–71.3	6.1(1.7)	33.9(9.8)
		gemma	cross	413(3)	67.4(1.4)	66.4–68.5	7.1(1.3)	38.3(4.9)
		qwen	cross	413(3)	53.9(1.9)	52.4–55.4	6.7(0.9)	43.4(5.9)

Alert-cut (the alert-count reduction against the frozen pool), CI, Prec. and Ret. are percentages; **Pool** is the frozen candidate count; **Prec.** is finding-level precision and **Ret.** the retention of detector-surfaced vulnerabilities, both under the mechanism-aware criterion at the precision-first threshold. Averaged over the four benchmarks, Nemotron attains the highest alert-cut on all three pools (55.2%, 61.0% and 55.3%), followed by gemma (39.5%, 39.3%, 54.0%) and qwen (36.8%, 33.6%, 48.1%); the ranking is identical on all three pools, so it is not an artifact of which model produced the candidates. Comparing gemma and qwen on their own pools against each other's pool isolates any self-verification effect: it is $+0.2$ percentage points for gemma and -3.2 for qwen, an order of magnitude smaller than the 18 to 27 point spread between verifiers on those two pools, though it varies by benchmark. Since the three verifiers' confidence intervals do not overlap on most cells, the verification stage is not model-agnostic.

contains no reentrant path. The verifier therefore judged the finding a false positive, discarding the hallucinated vulnerability while the genuine mechanism (the arbitrary call) was never the one reported.

B. CORRECTLY DOWNGRADING A FINDING

On Damn Vulnerable DeFi, a finding in FlashLoanReceiver.sol raised the concern that an unchecked block bypasses Solidity 0.8+ overflow protection and could cause an arithmetic overflow. The source code confirmed that the unchecked block is real—so the concern was not a hallucination—but inspection of the value ranges and calling context showed that the overflow cannot be triggered or exploited by an attacker, and no triggering PoC could be compiled. The verifier therefore downgraded the finding to low severity: neither dismissing a real code characteristic nor overstating it as a high-severity vulnerability.

C. AN INCORRECT VERDICT

On Damn Vulnerable DeFi, the puppet-v2 challenge, the detection stage correctly reported that the spot price can be manipulated through a flash loan, and the concern was sound. The verifier, however, reasoned incorrectly—concluding that the attack could only increase the collateral rather than yield a profit—and downgraded a genuine vulnerability. This illustrates a limitation of verification: for subtle oracle-manipulation logic, its reasoning can fail (see Section VI).

TABLE 11. Finding-level stability of the verification stage. Each frozen candidate pool is verified three times; *n* is the number of candidates that reach the verifier, Unan. the percentage on which all three passes make the same ship decision at the precision-first threshold, and κ Fleiss' kappa over the three passes. The first three blocks are the seed-1 pools of the main experiments (Table 2, panel (a)); the remaining blocks pool seeds 1–3 of the cross-verifier experiment (Table 10).

Pool	Benchmark	Verifier	n	Unan. (%)	κ
gemma	DVD	Nemotron	95	67.4	0.53
	PyGoat	Nemotron	104	68.3	0.54
	NodeGoat	Nemotron	38	97.4	0.96
	SmartBugs	Nemotron	565	69.7	0.59
qwen	DVD	Nemotron	112	83.9	0.77
	PyGoat	Nemotron	112	68.8	0.58
	NodeGoat	Nemotron	25	48.0	0.31
	SmartBugs	Nemotron	361	63.4	0.51
Nemotron	DVD	Nemotron	83	86.7	0.75
	PyGoat	Nemotron	93	88.2	0.83
	NodeGoat	Nemotron	39	97.4	0.97
	SmartBugs	Nemotron	411	84.4	0.76
gemma pool	DVD	gemma	274	94.9	0.93
	PyGoat	gemma	314	99.7	0.99
	NodeGoat	gemma	111	100.0	1.00
	SmartBugs	gemma	1703	95.5	0.94
gemma pool	DVD	qwen	274	96.4	0.95
	PyGoat	qwen	314	95.9	0.94
	NodeGoat	qwen	111	97.3	0.96
	SmartBugs	qwen	1703	97.2	0.96
qwen pool	DVD	gemma	336	97.0	0.96
	PyGoat	gemma	338	98.8	0.98
	NodeGoat	gemma	79	100.0	1.00
	SmartBugs	gemma	1070	96.6	0.96
qwen pool	DVD	qwen	336	95.5	0.94
	PyGoat	qwen	338	95.9	0.94
	NodeGoat	qwen	79	93.7	0.90
	SmartBugs	qwen	1070	98.9	0.98

VI. LIMITATIONS

1) Our recall measurement depends on mapping each finding to a vulnerability class via CWE metadata, and that mapping is incomplete on every benchmark: 46.8% of findings on Damn Vulnerable DeFi, 42.0% on PyGoat, 35.8% on NodeGoat, and 44.2% on SmartBugs carry no usable class label (Table 16). Mechanism-aware recall is therefore a lower bound throughout.
2) The verification layer is not perfect. For some subtle logic its reasoning can be wrong; for example, it misjudged an oracle-manipulation vulnerability in one of our case studies (Section V).
3) Any finding not in the documented vulnerability list—even one that is genuinely exploitable—is counted as a false positive. The reported false positive rate is therefore an upper bound and the reported precision a lower bound.
4) The PoC signal is not always available, and where it is available it is not always evidence about the target. On SmartBugs, PoC synthesis was attempted for 2994 candidates but none compiled (Table 1), and the Nemotron detection arm emits almost no PoCs on PyGoat and NodeGoat, so the contribution of the PoC verdict varies across benchmarks and detectors. On the web benchmarks the PoCs that do run are standalone scripts that re-implement the flagged function rather than requests against the running application, so a non-triggering verdict suppresses genuine findings at the precision-first point (RQ3).
5) We evaluate on only four benchmarks, a small sample size. Generalising our conclusions to more languages and to real-world projects will require validation on a broader range of benchmarks, and labelled ground truth is costly to construct: in a recent attempt to build such benchmarks by LLM-based vulnerability injection, only 32 of 193 distinct injected Solidity contracts survived validation [36].
6) The identity signal shows no measurable contribution under our settings: adding it changes the number of shipped findings by between -1.3 and $+3.0$, and its removal is significant in only 3 of the 12 model-benchmark cells (Tables 8 and 13). The two parts of the signal are redundant for different reasons. The function name already appears in the source code the verifier receives, on the signature line, so it duplicates information the verifier has. The file name does not appear there, but knowing which file a function lives in does not bear on whether the reported vulnerability is real. Identity nonetheless remains necessary to the pipeline, which uses it to locate and retrieve the source code.

VII. CONCLUSION

The problem we address is that LLMs hallucinate when used for vulnerability detection, producing false positives that bury genuine findings. We present a multi-signal focused verification stage that combines four signals—identity, source code, concern, and PoC verdict. The core principle is non-redundancy: each signal carries evidence the others do not, so one can correct another rather than reinforce a shared mistake, and we measure their pairwise agreement to confirm that they are neither interchangeable nor in agreement by chance. We also use targeted PoC-based execution as the source of the execution signal, admitting its verdict only when it carries information. Across four benchmarks spanning three different languages, the stage removes 54.6% to 64.6% of the shipped volume at the precision-first operating point in the main setting, while a recall-first operating point preserves substantially more recall—demonstrating a tunable precision–recall trade-off. A complete power-set ablation refines what each signal does: source and concern divide the labour between them along the language, with concern dominating on Solidity and source on Python and JavaScript, while the PoC verdict suppresses shipped findings at a recall cost and identity shows no measurable contribution to the verdict, remaining necessary only to locate the source code. Our key finding concerns generality: the stage reduces shipped findings in all 36 configurations we evaluate, so the architecture transfers, while the magnitude of its effect varies with three measurable factors—the breadth of detection, the availability of the PoC

TABLE 12. Complete power set of the four verification signals ($2^4 - 1 = 15$ non-empty subsets), qwen3-coder detection, precision-first threshold, mechanism-aware criterion. Entries are mean with standard deviation in parentheses, over five independent full-pipeline runs.

		DVD		PyGoat		NodeGoat		SmartBugs	
Subset	**Signals**	**TP**	**FP**	**TP**	**FP**	**TP**	**FP**	**TP**	**FP**
Full	I,S,C,P	1.4(1.1)	37.8(3.3)	7.8(2.5)	33.0(9.4)	3.0(2.5)	7.0(1.0)	10.2(2.3)	151.6(7.3)
−Identity	S,C,P	2.4(0.9)	38.0(7.3)	7.0(1.0)	23.8(5.5)	2.0(1.6)	6.6(0.9)	11.4(1.8)	155.4(4.0)
−Concern	I,S,P	1.4(1.1)	23.4(2.9)	8.4(1.9)	35.0(7.9)	3.0(3.0)	6.6(0.5)	7.8(1.5)	107.6(4.7)
−Source	I,C,P	2.2(0.8)	36.2(4.5)	3.6(1.3)	21.2(5.8)	0.2(0.4)	5.8(0.4)	15.8(1.6)	171.2(6.8)
−PoC	I,S,C	1.6(0.5)	32.2(3.5)	17.4(0.9)	47.6(2.3)	13.0(0.0)	8.4(0.9)	—	—
Keep I,S	I,S	1.4(0.5)	22.2(1.6)	19.8(0.8)	55.0(3.1)	13.0(0.0)	8.8(0.4)	8.0(3.9)	111.6(7.6)
Keep I,C	I,C	3.0(0.7)	40.2(5.0)	12.2(1.5)	33.2(3.6)	2.0(1.2)	7.2(0.4)	17.6(1.5)	178.8(12.7)
Keep I,P	I,P	0.4(0.5)	12.0(3.2)	4.0(2.3)	15.4(5.1)	0.2(0.4)	5.8(0.4)	—	—
Keep S,C	S,C	2.4(1.1)	33.4(5.5)	14.8(1.1)	41.0(1.7)	13.0(0.0)	8.0(0.0)	10.8(1.3)	154.6(5.6)
Keep S,P	S,P	1.2(0.4)	24.6(5.9)	9.2(2.6)	37.0(8.1)	2.6(2.6)	6.8(0.4)	—	—
Keep C,P	C,P	1.2(0.8)	34.4(1.7)	4.8(1.8)	20.6(6.3)	0.4(0.9)	6.0(0.0)	—	—
Only I	I	0.0(0.0)	2.2(0.4)	0.4(0.5)	2.0(0.7)	0.2(0.4)	5.8(0.4)	3.0(1.2)	29.6(4.3)
Only S	S	1.4(1.1)	22.4(4.2)	20.2(0.4)	55.0(3.7)	13.0(0.0)	9.2(0.4)	8.0(1.6)	118.8(10.5)
Only C	C	2.6(1.1)	37.4(3.6)	9.8(1.8)	29.8(3.3)	2.2(1.3)	7.4(0.5)	15.4(2.9)	172.6(6.8)
Only P	P	0.2(0.4)	11.6(3.0)	3.4(1.8)	16.0(5.0)	0.2(0.4)	5.8(0.4)	—	—

I = identity, S = source code, C = concern, P = PoC verdict. TP and FP are counts of shipped findings. Subsets containing P are omitted for SmartBugs, where that signal is structurally unavailable (Table 1); the SmartBugs power set therefore has $2^3 - 1 = 7$ subsets.

TABLE 13. Finding-level paired significance tests. For each ablation, findings shipped under exactly one of the two configurations are counted and compared with an exact binomial (McNemar) test, pooled over five runs. Holm correction is applied within each model–benchmark cell.

Model	**Bench.**	**Abl.**	**Full**	**Abl.**	n	p_{Holm}	**Model**	**Bench.**	**Abl.**	**Full**	**Abl.**	n	p_{Holm}
gemma	DVD	−Id	62	50	112	0.597	qwen	NodeGoat	−Id	17	10	27	0.496
		−Con	119	30	149	3.8e−13*			−Con	12	10	22	0.832
		−Src	94	47	141	2.8e−04*			−Src	20	0	20	5.7e−06*
		−PoC	56	54	110	0.924			−PoC	2	59	61	6.6e−15*
	PyGoat	−Id	96	50	146	3.5e−04*		SmartBugs	−Id	209	234	443	0.508
		−Con	66	54	120	0.315			−Con	393	161	554	8.7e−23*
		−Src	153	7	160	2.7e−36*			−Src	248	374	622	1.5e−06*†
		−PoC	25	181	206	6.7e−30*			−PoC	0	0	0	1.000
	NodeGoat	−Id	10	3	13	0.185	Nemotron	DVD	−Id	20	22	42	1.000
		−Con	8	4	12	0.388			−Con	60	31	91	9.3e−03*
		−Src	82	0	82	1.7e−24*			−Src	54	21	75	7.1e−04*
		−PoC	2	11	13	0.067			−PoC	25	25	50	1.000
	SmartBugs	−Id	419	424	843	1.000		PyGoat	−Id	41	16	57	3.8e−03*
		−Con	620	403	1023	4.9e−11*			−Con	26	38	64	0.337
		−Src	535	478	1013	0.235			−Src	187	20	207	1.4e−34*
		−PoC	0	0	0	1.000			−PoC	20	20	40	1.000
qwen	DVD	−Id	40	46	86	1.000		NodeGoat	−Id	2	0	2	1.000
		−Con	111	39	150	1.4e−08*			−Con	0	2	2	1.000
		−Src	71	67	138	1.000			−Src	74	0	74	4.2e−22*
		−PoC	69	42	111	0.040*†			−PoC	2	0	2	1.000
	PyGoat	−Id	87	37	124	1.7e−05*		SmartBugs	−Id	112	138	250	0.227
		−Con	36	49	85	0.193			−Con	264	154	418	3.3e−07*
		−Src	102	22	124	5.0e−13*			−Src	260	151	411	3.3e−07*
		−PoC	36	157	193	1.3e−18*			−PoC	0	0	0	1.000

−Id = −Identity, −Con = −Concern, −Src = −Source. **Full** is the number of findings shipped only under the full configuration; **Abl.** only under the ablated one; n is their sum. * significant at $\alpha = 0.05$ after Holm correction. † the effect is significant but in the direction opposite to that predicted: for −Identity, −Concern and −Source the expected direction is a decrease in shipped findings, whereas for −PoC it is an increase, since a non-triggering PoC verdict suppresses a spurious finding (Section III-D). Only two of the 48 cells show a significant reversal. Pairing at the finding level draws power from hundreds of paired candidates rather than from the number of runs, so five seeds suffice.

verdict, and the capability of the verifier. The strongest verifier removes the most volume but retains the fewest detector-surfaced vulnerabilities, so the choice of verification model is a precision–recall trade-off.

Several directions remain for future work. Detection sets the ceiling on end-to-end recall—particularly for subtle semantic vulnerabilities—but a stronger detection model does not straightforwardly improve the outcome, since the model that surfaces the fewest candidates on one benchmark yields the largest reduction there and the smallest on another. Evaluating more benchmarks and real-world projects would strengthen the evidence for generality, and refining the

TABLE 14. Pairwise agreement between single-signal verdicts. Each single-signal configuration runs the verifier with exactly one signal, so the verdicts are behavioural measurements rather than model self-reports.

Benchmark	Pair	n	κ	ϕ	Agr. (%)
DVD	I–S	456	—	—	—
	I–C	456	—	—	—
	I–P	456	—	—	—
	S–C	456	0.194	0.205	63.2
	S–P	456	0.080	0.104	72.8
	C–P	456	0.036	0.060	59.2
PyGoat	I–S	341	0.014	0.084	45.2
	I–C	341	−0.002	−0.010	61.0
	I–P	341	−0.017	−0.042	82.7
	S–C	341	0.314	0.333	64.8
	S–P	341	−0.046	−0.067	43.7
	C–P	341	0.101	0.119	61.9
NodeGoat	I–S	49	0.024	0.110	38.8
	I–C	49	0.082	0.207	69.4
	I–P	49	−0.021	−0.021	95.9
	S–C	49	0.439	0.531	69.4
	S–P	49	0.024	0.110	38.8
	C–P	49	0.082	0.207	69.4
SmartBugs	I–S	1256	0.114	0.165	61.7
	I–C	1256	0.086	0.161	48.3
	S–C	1256	0.303	0.320	64.2

I = identity, S = source code, C = concern, P = PoC verdict. κ is Cohen's kappa and ϕ the Matthews correlation coefficient. Values bounded away from both 0 and 1 indicate that the signals are correlated but non-redundant. On DVD, identity alone yields a constant verdict, so κ is undefined for the three pairs involving it; this is itself evidence that identity carries no independent discriminative power. Pairs involving P are absent for SmartBugs, where the signal is unavailable.

vulnerability-class mapping, on which mechanism-aware recall depends and which leaves between 35.8% and 46.8% of findings unmappable, would give a more precise picture of the system's behaviour.

APPENDIX A
COMPLETE SIGNAL ABLATION

Table 12 reports all $2^4 - 1 = 15$ non-empty subsets of the four verification signals, and Table 13 the corresponding finding-level paired significance tests. These are given in full so that the marginal contributions summarised in Table 8 can be recomputed independently.

APPENDIX B
SIGNAL AGREEMENT

Table 14 reports pairwise agreement between the single-signal verdicts. Because each single-signal configuration runs the verifier with exactly one signal, these are behavioural measurements rather than model self-reports, and they are the basis for the non-redundancy claim in Section III-D.

APPENDIX C
VERIFICATION PROMPT

The verifier receives a single prompt assembled from the signals admitted by the configuration. With all four signals the prompt is, verbatim:

TABLE 15. Mapping from CWE identifiers to benchmark vulnerability classes, used for mechanism-aware matching. A CWE may map to several classes where the benchmark distinguishes variants that the CWE does not.

CWE	Benchmark class
Damn Vulnerable DeFi	
284, 285	access_control, access_control_upgrade, access_control_calldata
841	reentrancy
400	denial_of_service
807	oracle_manipulation
665	access_control_upgrade
190, 682	logic_rounding, logic_price
681	logic_rounding
837, 294	message_replay
367	governance_flashloan
754	arbitrary_call
PyGoat	
89	sql_injection
78, 94, 95	command_injection
502	insecure_deserialization
611	xxe
79, 80	xss
918	ssrf
565, 639, 668, 784	broken_access_control
NodeGoat	
94, 95	ssji
89, 943	nosql_injection
256, 327, 522	weak_crypto
400, 1333	redos
284, 639	broken_access_control
79, 116	xss
918	ssrf
SmartBugs (DASP-10)	
284, 285, 347	access_control
841	reentrancy
190, 191, 681, 682	arithmetic
400	denial_of_service
20, 252, 778	unchecked_low_level_calls
330, 338, 341	bad_randomness
367	front_running, time_manipulation
294	time_manipulation

Where no CWE is available, the label falls back to the query type of the L5 proactive category scan, one of: *reentrancy*, *access_control*, *arbitrary_call*, *oracle_manipulation*, *flashloan*, *insecure_crypto*, *ssrf*, *xss*, *sqli*, *deserialization*, *arithmetic*, *randomness*, *dos*. Findings matching neither source are counted as unmappable (Table 16).

TABLE 16. Provenance of the vulnerability-class label used for mechanism-aware matching. Counts are pooled over all configurations and runs.

Benchmark	Findings	CWE	L5 query	Unmap.	%
DVD	58 024	29 815	1 069	27 140	46.8
PyGoat	29 963	16 370	1 000	12 593	42.0
NodeGoat	13 921	7 685	1 250	4 986	35.8
SmartBugs	213 775	118 759	541	94 475	44.2

Only machine-generated sources are used: the CWE metadata emitted by the static scanner (Table 15), and the query type of the L5 proactive category scan. No keyword matching or manual assignment is applied, so the unmappable fraction is *reported* rather than reduced. Mechanism-aware recall is therefore a lower bound.

```
A global security pattern analysis flagged
this function.
File: {file}  Function: {function}
Flagged concern: {concern}
```

```
Actual source code: {snippet}
≡ PoC Validation Signal ≡
{runner} build: PASS | FAIL -- PoC could
not be compiled/run
{runner} run: TRIGGERED expected exploit
(confidence {c}) | PoC did NOT trigger
expected outcome -- the flagged
vulnerability may not be exploitable as
described
Based on the evidence above, choose one
verdict: "confirmed": the vulnerability is
real and exploitable at the flagged
severity; "downgraded": there is some
merit (defense-in-depth, theoretical risk)
but not exploitable at the flagged
severity; "false_positive": the concern is
invalid, the code is safe.
Weight the PoC signal heavily --
poc_triggers_bug=true with a
ground_truth_confidence ≥ 0.7 is strong
evidence FOR the finding.
poc_triggers_bug=false is evidence
AGAINST. PoC compile failure means no
ground truth -- fall back to source-only
judgment.
Respond with strict JSON: {"verdict":
"confirmed"|"downgraded"|"false_positive",
"severity": "critical"|"high"|"medium"
|"low"|"info", "reason": "one sentence"}
```

Removing a signal removes its block; the PoC block is present only when a PoC verdict exists for the function. The complete prompt assembly and the PoC runner are in the replication package.

APPENDIX D
VULNERABILITY-CLASS LABELLING

Table 15 gives the mapping from CWE identifiers to benchmark vulnerability classes used for mechanism-aware matching, and Table 16 the resulting label provenance. The mapping is the only judgment entering the mechanism-aware criterion and is therefore reported in full.

DATA AVAILABILITY

The replication package—evaluation harness, experiment and analysis scripts, raw per-run outputs of every configuration, adjudication items with both judges' labels and justifications, and scripts that regenerate every table—is archived at Zenodo: https://doi.org/10.5281/zenodo.22704663.

ACKNOWLEDGMENT

The author thanks Wenyi Tsai for proofreading and helpful comments on earlier drafts of this manuscript. The author is also grateful to Matthew Tseng, whose early mentorship in mathematics and programming first sparked the author's path toward computer security.

During the preparation of this work, the author used Claude Opus 4.8 and Claude Fable 5.1 (Anthropic) in three capacities. First, they were used as coding assistants to help generate the code used in this work (the pipeline of Section III and the experiment and analysis scripts of Section IV); all such code was reviewed, tested, and validated by the author, and all reported experimental results were produced by the author's own execution of this code and verified manually. Second, the two models served as the independent judges in the adjudication study of Section IV-E (RQ1, Table 7), as described there; they were not used in the detection or verification pipeline under evaluation. Third, they were used to assist with language editing (grammar and clarity) of author-written text. The author reviewed and edited all content and takes full responsibility for the content of this work.

REFERENCES

[1] Z. Xiao, Q. Wang, H. Pearce, and S. Chen, "Logic meets magic: LLMs cracking smart contract vulnerabilities," 2025, *arXiv:2501.07058*.

[2] Z. Wei, J. Sun, Z. Zhang, X. Zhang, X. Yang, and L. Zhu, "Survey on quality assurance of smart contracts," *ACM Comput. Surv.*, vol. 57, no. 2, pp. 1–36, Feb. 2025, doi: 10.1145/3695864.

[3] A. Gervais and L. Zhou, "AI agent smart contract exploit generation," 2025, *arXiv:2507.05558*.

[4] B. Aloraini, M. Nagappan, D. M. German, S. Hayashi, and Y. Higo, "An empirical study of security warnings from static application security testing tools," *J. Syst. Softw.*, vol. 158, Dec. 2019, Art. no. 110427, doi: 10.1016/j.jss.2019.110427.

[5] A. Lekssays, H. Mouhcine, K. Tran, T. Yu, and I. Khalil, "LLMxCPG: Context-aware vulnerability detection through code property graph-guided large language models," in *Proc. 34th USENIX Secur. Symp.*, Seattle, WA, USA, 2025, pp. 489–507.

[6] H. Wu, H. Wang, S. Li, Y. Wu, M. Fan, Y. Zhao, and T. Liu, "Detecting state manipulation vulnerabilities in smart contracts using LLM and static analysis," 2025, *arXiv:2506.08561*.

[7] C. Wen, Y. Cai, B. Zhang, J. Su, Z. Xu, D. Liu, S. Qin, Z. Ming, and C. Tian, "Automatically inspecting thousands of static bug warnings with large language model: How far are we?" *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 7, Aug. 2024, Art. no. 168, doi: 10.1145/3653718.

[8] X. Du, K. Yu, C. Wang, Y. Zou, W. Deng, Z. Ou, X. Peng, L. Zhang, and Y. Lou, "Minimizing false positives in static bug detection via LLM-enhanced path feasibility analysis," 2025, *arXiv:2506.10322*.

[9] X. Du, J. Feng, Y. Zou, W. Xu, J. Ma, W. Zhang, S. Liu, X. Peng, and Y. Lou, "Reducing false positives in static bug detection with LLMs: An empirical study in industry," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng., Softw. Eng. Pract.*, Apr. 2026, pp. 716–727, doi: 10.1145/3786583.3786910.

[10] M. R. Ameen, M. T. Ul Alam, and A. Islam, "QASecClaw: A multi-agent LLM approach for false positive reduction in static application security testing," 2026, *arXiv:2605.01885*.

[11] Y. Xiong and T. Zhang, "Sifting the noise: A comparative study of LLM agents in vulnerability false positive filtering," 2026, *arXiv:2601.22952*.

[12] S. Xia, S. Shao, M. He, T. Yu, L. Song, and Y. Zhang, "AuditGPT: Auditing smart contracts with ChatGPT," 2024, *arXiv:2404.04306*.

[13] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. IEEE/ACM 2nd Int. Workshop Emerg. Trends Softw. Eng. Blockchain (WETSEB)*, May 2019, pp. 8–15.

[14] Semgrep. *Semgrep: Static Analysis at Lightning Speed*. Accessed: Oct. 3, 2026. [Online]. Available: https://semgrep.dev

[15] W. Haouari, A. Senhaji Hafid, and M. Fokaefs, "Vulnerabilities of smart contracts and mitigation schemes: A comprehensive survey," 2024, *arXiv:2403.19805*.

[16] G. Bennett, T. Hall, E. Winter, and S. Counsell, "Semgrep*: Improving the limited performance of static application security testing (SAST) tools," in *Proc. 28th Int. Conf. Eval. Assessment Softw. Eng.*, Salerno, Italy, Jun. 2024, pp. 614–623, doi: 10.1145/3661167.3661262.

[17] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in software security: A survey of vulnerability detection techniques and insights," 2025, *arXiv:2502.07049*.

[18] P. Ince, J. Yu, J. K. Liu, and X. Du, "Generative large language model usage in smart contract vulnerability detection," 2025, *arXiv:2504.04685*.

[19] Z. Wei, J. Sun, Y. Sun, Y. Liu, D. Wu, Z. Zhang, X. Zhang, M. Li, Y. Liu, C. Li, M. Wan, J. Dong, and L. Zhu, "Advanced smart contract vulnerability detection via LLM-powered multi-agent systems," *IEEE Trans. Softw. Eng.*, vol. 51, no. 10, pp. 2830–2846, Oct. 2025, doi: 10.1109/TSE.2025.3597319.

[20] Z. Wei, J. Sun, Z. Zhang, X. Zhang, M. Li, and Z. Hou, "LLM-SmartAudit: Advanced smart contract vulnerability detection," 2024, *arXiv:2410.09381*.
[21] Z. Wei, J. Sun, Z. Zhang, Z. Hou, and Z. Zhao, "Adaptive plan-execute framework for smart contract security auditing," 2025, *arXiv:2505.15242*.
[22] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2025, doi: 10.14722/ndss.2025.241357.
[23] S. J. Wang, K. Pei, and J. Yang, "SmartInv: Multimodal learning for smart contract invariant inference," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 2217–2235, doi: 10.1109/SP54263.2024.00126.
[24] S. Chen, T. Luo, S. Wu, X. Liu, Y. Zhou, Q. Li, and W. Xu, "A multi-agent framework for automated exploit generation with constraint-guided comprehension and reflection," 2026, *arXiv:2604.05130*.
[25] The Red Guild. *Damn Vulnerable DeFi (v4)*. Accessed: Oct. 3, 2026. [Online]. Available: https://www.damnvulnerabledefi.xyz
[26] OWASP. *OWASP NodeGoat Project*. Accessed: Oct. 3, 2026. [Online]. Available: https://owasp.org/www-project-node.js-goat/
[27] OWASP. *OWASP PyGoat*. Accessed: Oct. 3, 2026. [Online]. Available: https://owasp.org/www-project-pygoat/
[28] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "SmartBugs: A framework to analyze solidity smart contracts," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Melbourne, VIC, Australia, Sep. 2020, pp. 1349–1352.
[29] Y. Song, G. Wang, S. Li, and B. Y. Lin, "The good, the bad, and the greedy: Evaluation of LLMs should not ignore non-determinism," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL-HLT 2025)*, 2025, pp. 4195–4206.
[30] M. Astekin, M. Hort, and L. Moonen, "An exploratory study on how non-determinism in large language models affects log parsing," in *Proc. ACM/IEEE 2nd Int. Workshop Interpretability, Robustness, Benchmarking Neural Softw. Eng.*, Apr. 2024, pp. 13–18.
[31] H. Tamba, "Necessary but not sufficient: Temperature control and reproducibility in LLM-as-judge safety evaluations," 2026, *arXiv:2606.26185*.
[32] L. Li, L. Sleem, Y. Xu, Y. Song, A. Jia, J. Francois, and R. State, "The necessity of setting temperature in LLM-as-a-judge," 2026, *arXiv:2603.28304*.
[33] F. Angermeir, M. Amougou, M. Kreitz, A. Bauer, M. Linhuber, D. Fucci, F. Moyón Constante, D. Mendez, and T. Gorschek, "Reflections on the reproducibility of commercial LLM performance in empirical software engineering studies," 2025, *arXiv:2510.25506*.
[34] Y. Sun, D. Wu, Y. Xue, H. Liu, W. Ma, L. Zhang, Y. Liu, and Y. Li, "LLM4Vuln: A unified evaluation framework for decoupling and enhancing LLMs' vulnerability reasoning," 2024, *arXiv:2401.16185*.
[35] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 862–880.
[36] L. Migliaccio, R. Natella, N. Ivaki, N. Laranjeiro, and M. Vieira, "Automated vulnerability injection in smart contracts using large language models," 2026, *arXiv:2609.02624*.

CASPER CHEN is a Security Researcher based in New Taipei City, Taiwan. Chen's research interests include software security, large language models, and their application to automated vulnerability detection.